



CCS3440 – ARTIFICIAL INTELLIGENCE

Machine Learning-Based Low, Medium, and High Disease Risk Classification Using Patient Clinical Data

Option C – Disease Risk Classification (Multi-Class Classification)

Team members:

CIT-23-02-0173-S.A.Thilani Dilmani

CIT-23-02-0044-G.Ishini Sivod

CIT-23-02-0153-R.M Nuwani Umanda

CIT-23-02-0159- Volga Indeewari

GitHub Repository: [Git Repo Link](#)

Presentation: [Presentation-Google Drive](#)

Video Demonstration: [Video Demonstration-Google Drive](#)

Prototype Source Code: [Prototype-Google Drive](#)

Jupyter Notebook: [Jupyter Notebooks-Google Drive](#)

Train Model Files: [Trained Model Files-Google Drive](#)

Table of Contents

1. Introduction.....	3
2. Problem Definition.....	3-4
3. Literature Review.....	4-7
4. Research Gap Analysis.....	8
5. Dataset Description.....	8-13
6. Data Preprocessing and Feature Engineering.....	13-22
7. Exploratory Data Analysis.....	23-28
8. Machine Learning Model Development.....	28-36
9. Model Evaluation.....	36-41
10. Explainable AI Analysis.....	42-47
11. AI Prototype Development.....	47-49
12. Discussion & Results.....	50
13. Conclusion.....	50
14. References.....	51

1. Introduction

Artificial intelligence (AI) and machine learning are increasingly used in healthcare to analyze complex data about patients and detect diseases early. Demographic data, vital signs, laboratory measurements, medical history and lifestyle facts are all routinely collected by hospitals. When these variables are considered together, it can reveal patterns that are difficult to identify through manual inspection alone. Early risk recognition is important because chronic conditions such as diabetes, cardiovascular disease, chronic kidney disease and fatty liver disease may progress before obvious symptoms appear.

This project demonstrates a supervised machine learning system using the SmartCare Hospital AI dataset. This dataset consists of 1,000 patient records and 33 attributes covering demographics, appointments, admissions, clinical measurements, treatments and billing. Relevant clinical and demographic factors are used for classifying patients as having low, medium, or high risk of contracting diseases. The resulting system is intended to support preventive healthcare and can be used for prioritization, and is explicitly a decision-support aid rather than a diagnostic replacement for the judgement of qualified clinicians.

This report documents the complete AI project lifecycle followed for the coursework: problem definition and literature review, dataset understanding, data preprocessing and feature engineering, exploratory data analysis, building several machine learning models, comparing their performance, explaining how the best model makes its decisions, and finally building a simple prototype that a hospital staff member could use.

2. Problem Definition

2.1 Background and Problem Statement

Disease risk is influenced by interactions among age, body mass index (BMI), blood pressure, blood glucose, cholesterol, existing conditions, family history, and health behaviors. A single measurement rarely represents overall health status. Manual review of many variables across large patient populations can be slow, inconsistent and vulnerable to overlooked relationships. Traditional binary decisions such as “Disease” or “No Disease” also provide limited support for differentiating patients who require different levels of preventive attention.

The problem addressed in this study is therefore the need for a consistent, data-driven, and interpretable method that combines multiple routinely gathered patient attributes and assigns a clinically understandable risk category. Low-risk patients may require maintenance of healthy

behaviors, medium-risk patients may benefit from monitoring and lifestyle intervention, and high-risk patients may require prompt professional assessment.

2.2 Aim and Objectives

The aim is to develop and evaluate an explainable multi-class machine learning model that predicts Low, Medium or High disease risk from SmartCare patient data. The specific objectives are:

- Examine dataset structure, data types, quality, and target-class distribution of the SmartCare dataset.
- Clean missing, duplicate, inconsistent, and outlying values and encode categorical features.
- Determine relevant demographic, clinical, and operational predictors.
- Conduct a Compare Logistic Regression, Decision Tree, Random Forest, XGBoost, Voting Ensemble, and Stacking Ensemble models.
- Evaluate model accuracy, macro- and weighted-average precision, recall and F1-score, and confusion matrices, including class-level results.
- Interpret the selected model using SHAP and LIME.
- Build and design a simple prediction prototype for hospital staff.

2.3 Significance

An accurate three-layer classifier could help to make interventions earlier, improve prioritization of limited healthcare resources, and provide consistent screening for preliminary diagnoses. Explainability is essential because clinicians must understand which factors contribute to a prediction. Responsible use also demands protection of privacy, awareness of class imbalances, and clear communication that model outputs are advisory rather than diagnostic.

3. Literature Review

A total of eight articles and research papers have been reviewed because of the application of the supervised machine learning process to patient information for either diagnosis or risk stratification. This includes papers addressing diabetes, prediabetes, cardiovascular disease, and chronic kidney disease. Below is a brief synopsis of each study as well as an analysis of what these studies collectively reveal.

Topic	Study	Method/Data	Key Findings	Main Limitation
Transparent Multiclass Diabetes Prediction	Niloufar Zaferani; Mohammad Reza Afrash; Khadijeh Moulaei	768 Pima Indians Diabetes records, 8 features (glucose, BMI, blood pressure, insulin, age, family history); stacking/soft-voting ensemble of Neural Network, KNN, and Random Forest, with Logistic Regression as meta-model; LIME and SHAP applied.	Closest study to our Low/Medium/High design patients relabeled as Normal, Prediabetes, or Diabetes. The stacking model gave the strongest performance; glucose, BMI, and blood pressure were the dominant features.	Narrow population, small sample. Glucose was used both to build the class labels and kept as an input, a possible outcome-leakage issue that may inflate results.
Non-Invasive MAFLD Identification	Yaxuan He; Yu Cao; Zekai Chen; Rong Xiang; Fang Wang	23,348 adults from a Chinese hospital cohort, externally tested on a further 3,357 participants; 13 body-composition/demographic variables; 8 algorithms with 10-fold cross-validation, calibration, and SHAP.	Tree-based models (XGBoost, Gradient Boosting, LightGBM) reached internal AUC above 0.96 and external AUC above 0.95 for fatty liver disease (MAFLD). Visceral-fat rating, waist circumference, and BMI were the strongest predictors.	Single-center, cross-sectional; body-fat estimate less accurate than CT/MRI; output was a binary MAFLD label, not a graded risk.
Type 2 Diabetes in the Henan Rural Cohort	Liyang Zhang; Yikang Wang; Miaomiao Niu; Chongjian Wang; Zhenfei Wang	36,652 rural Chinese participants; Logistic Regression, CART, ANN, SVM, Random Forest, and Gradient Boosting compared with and without laboratory data, for Type 2 diabetes risk.	Gradient Boosting performed best (AUC 0.872 with lab data, 0.817 without). Urine glucose, age, heart rate, waist circumference, and hypertension were key predictors.	Cross-sectional design prevents temporal inference; no external validation; rural population may not generalize elsewhere.
Interpretable Prediabetes Risk Prediction	Xiaolong Li; Fan Ding; Lu Zhang; Shi Zhao; Zengyun Hu; Zhanbing Ma; Feng Li; Yuhong Zhang; Yi Zhao; Yu Zhao	8,277 participants from the China Health and Nutrition Survey (7.13% prediabetes prevalence), externally checked on NHANES; LASSO selected 9 factors; 7 algorithms compared after oversampling.	XGBoost achieved AUC 0.939, accuracy 0.845, sensitivity 0.896, F1-score 0.453. Logistic Regression had higher accuracy but very low sensitivity. Showing accuracy alone can mislead on imbalanced data.	Regional data only; no genetic factors included; cross-sectional design limits causal and population-wide conclusions.

Diabetes and Cardiovascular Disease Prediction	An Dinh; Stacey Miertschin; Amber Young; Somya D. Mohanty	Multiple NHANES cycles, laboratory and non-laboratory variants; Logistic Regression, SVM, Random Forest, XGBoost, and a weighted ensemble compared after down-sampling, for diabetes/prediabetes/cardiovascular risk.	XGBoost reached AUROC 0.957 with lab data (0.862 without) for diabetes. Prediabetes reached 0.737 without laboratory data and 0.844 with laboratory data. The cardiovascular ensemble reached ~0.831. Age, waist size, and blood pressure were influential.	Detects present status rather than future onset; no independent prospective clinical deployment shown.
Cardiovascular Diagnosis Using Combined Data	Khandaker Mohammad Mohi Uddin; Rokaiya Ripa; Nilufar Yeasmin; Nitish Biswas; Samrat Kumar Dey	Three combined heart-disease datasets: ten classifiers compared (Decision Tree, AdaBoost, Extra Trees, SVM, Gradient Boosting, XGBoost, Random Forest, KNN, Logistic Regression), with a web app demo.	Decision Tree reached 99.16% accuracy.	Combining datasets from different sources can introduce duplicates or leakage; without a separate external cohort, the near-perfect score needs caution.
Explainable CKD Detection	Mc Neil Valencia; Jun Kim; Zeeshan Abbas; Seung Won Lee	968 NHANES men aged 30–60: demographic, biochemical, lifestyle, dietary, sleep, and activity factors; Random Forest, AdaBoost, Naive Bayes, SVM, and XGBoost compared for chronic kidney disease.	AdaBoost gave the most balanced result (accuracy ~0.726, recall 0.6923, F1 0.6457, PR-AUC 0.715 and ROC-AUC 0.785). serum creatinine, blood urea nitrogen, urinary creatinine, and age were dominant predictors.	Male-only, middle-aged sample; self-reported lifestyle variables; cross-sectional design restricts generalization.
Diabetes Risk Among Patients with MASLD	Sijie Yang; Ruiqing Mo; Wenqiang Wang; Puxiang Zhen; Wentian Han	6,310 NHANES participants with MASLD; LASSO/Random Forest/SVM-RFE selected 8 predictors; 9 models compared with SMOTE balancing, for diabetes risk.	LightGBM was selected for its balanced overall performance and SHAP interpretability.	Cross-sectional target reflects current status, not future onset; no independent external validation; findings specific to MASLD patients.

Table 1: Literature Summary

3.1 Synthesis of Existing AI Approaches

Supervised classification is the dominant approach across the literature. Logistic Regression is typically preferred as an interpretable baseline method, while Decision Tree, SVM and KNN provide alternative decision structures. Random Forest and boosting methods usually achieve stronger discrimination because they represent nonlinear interactions and reduce single-model variance. Nowadays, the interpretation of features' impact becomes possible using SHAP or LIME.

A typical data processing workflow consists of missing value imputation, categorical encoding, scaling, feature selection, class balancing, model training, hyperparameter tuning, and testing on holdout data or cross-validation. When dealing with health care problems, recall and class-level F1 are essential because a model can achieve high accuracy while failing to identify high-risk minority cases.

3.2 Current Limitations in the Literature

1. **Limited generalizability** – Most studies rely on a single country, hospital, or restricted demographic group, so findings don't generalize well.
2. **Mislabeled prediction task** – Cross-sectional data are often described as "risk prediction," even though they actually classify present disease status rather than predicting future risk.
3. **Lack of external validation** – Models are rarely tested on independent, external datasets.
4. **Overfitting risk** – Small or merged datasets increase the likelihood of overfitting.
5. **Class imbalance masking performance** – Reporting accuracy alone can hide poor performance on minority classes when the dataset is imbalanced.
6. **Interpretability barrier** – Ensemble and neural network models remain difficult to interpret in practice.
7. **Data leakage** – Some studies include variables used to define the outcome as predictors, artificially inflating performance.
8. **Narrow scope and lack of deployment** – Most studies:
 - Predict only a single disease as a binary outcome, and
 - Stop at experimental/offline evaluation, without producing a clinically usable, privacy-aware decision-support prototype.

4. Research Gap Analysis

The first gap is the limited availability of transparent multiclass systems that combine routinely available patient attributes to classify general disease risk as Low, Medium or High. Most previous studies focus on Disease/No-Disease for a single condition and therefore do not directly support graded prioritization across a broader patient population.

The second gap arises from the lack of comparison of algorithms on one consistently processed dataset, since different studies use different populations, targets, feature sets, and validation strategies. A third gap concerns the balance between performance and interpretability; although sophisticated ensemble models achieve very high accuracy, their deployment in healthcare requires an explanation of how the model makes decisions. Finally, the issues of imbalance, target leakage, privacy, fairness, and real-world usability are not addressed consistently.

The present study addresses these deficiencies through the comparison of several classifiers for the same SmartCare data, the use of multiple performance measures for multiclass problems, and model explanations, while the final classifier is presented as a decision-supporting tool instead of a diagnostic one.

5. Dataset Description

5.1 Dataset Overview

The smartcare_ai_dataset_1000.csv contains 1,000 hospital entries and 33 attributes, along with a data dictionary, smartcare_ai_dataset_data_dictionary.csv, which gives information about each column.

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (8, 5)
RANDOM_STATE = 42

df_raw = pd.read_csv('/content/drive/MyDrive/SmartCare/smartcare_ai_dataset_1000.csv')
data_dict = pd.read_csv('/content/drive/MyDrive/SmartCare/smartcare_ai_dataset_data_dictionary.csv')
df = df_raw.copy() # all transformations happen on this working copy; df_raw stays untouched for viva reference
df.shape

print(f"Records: {df.shape[0]}, Columns: {df.shape[1]}")
df.head()
```

Records: 1000, Columns: 33


```
# Data dictionary is the authoritative description of every column
pd.set_option('display.max_rows', None)
data_dict
```

Figure 1: Loading the dataset

The dataset covers four broad groups of information:

patient demographics, clinical measurements, hospital operations, and financial/billing data. It was also designed to support two alternative AI prediction targets, no_show (appointment no-show) and readmitted_30_days (30-day readmission), alongside the disease_risk_level target selected for this project as an option C.

Category	Representative Attributes
Patient information	patient_id, age, gender, blood_group
Clinical information	diagnosis, systolic_bp, diastolic_bp, blood_sugar_mg_dl, cholesterol_mg_dl, bmi
Hospital operations	department, appointment_date, waiting_days, previous_appointments, admitted, room_type, length_of_stay_days, previous_admissions, lab_tests_count, treatments_count
Financial data	consultation_fee_lkr, room_charge_lkr, lab_charge_lkr, medicine_charge_lkr, total_bill_lkr, payment_status, payment_method
AI targets	no_show, readmitted_30_days, disease_risk_level (selected target)

Table 2: Attribute groups in the SmartCare Hospital AI dataset (33 attributes total).

5.2 Structural Overview

df.info() was used to inspect data types and non-null counts in a single pass, the fastest way to spot columns of the wrong type or columns that already show missing values.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 33 columns):
#   Column                               Non-Null Count  Dtype  
---  --
0   record_id                            1000 non-null   int64   
1   patient_id                           1000 non-null   object  
2   age                                  1000 non-null   int64   
3   gender                               1000 non-null   object  
4   blood_group                          1000 non-null   object  
5   department                           1000 non-null   object  
6   diagnosis                            1000 non-null   object  
7   appointment_date                     1000 non-null   object  
8   waiting_days                         1000 non-null   int64   
9   previous_appointments                1000 non-null   int64   
10  missed_previous_appointments          1000 non-null   int64   
11  appointment_status                    1000 non-null   object  
12  admitted                             1000 non-null   int64   
13  room_type                             94 non-null     object  
14  length_of_stay_days                   1000 non-null   int64   
15  previous_admissions                   1000 non-null   int64
```

Figure 2:Dataset Structure Overview

Property	Value
Total records	1,000
Total columns	33
Column with missing values	room_type (94 non-null / 906 missing)
Duplicate rows	0

Table 3: Dataset Overview

5.3 Selected Target Variable

Based on that, disease_risk_level has been chosen as the target variable. This is a multi-class target variable with three classes: Low, Medium, and High. However, it can be seen from Table 4 below that there is a problem of imbalance in the dataset, with the Low-risk category greatly under-represented relative to Medium and High risk. This imbalance was carried forward into model training and evaluation decisions in later sections.

Risk Class	Record Count	Proportion
Medium	469	46.9%
High	400	40.0%
Low	131	13.1%
Total	1,000	100%

Table 4: Class distribution of the disease_risk_level target.

5.4 Summary Statistics

Descriptive statistics such as count, mean, std, min, quartiles, and max were generated for all numeric columns using `df.describe()`. Age (1–90), BMI (~14–39), blood pressure, and cholesterol ranges all look clinically plausible. `waiting_days`, `charges`, and `length_of_stay_days` are right-skewed (mean greater than median), which is expected for cost/time data and is revisited with histograms in Task 04.

Column	Mean	Std	Min	Median	Max
age	44.74	17.85	1.0	44.0	90.0
systolic_bp	128.42	15.49	85.0	128.0	178.0
diastolic_bp	78.83	10.04	50.0	79.0	111.0
blood_sugar_mg_dl	114.83	26.02	65.0	114.0	201.0
cholesterol_mg_dl	204.81	36.65	100.0	204.0	330.0
bmi	25.52	4.25	14.0	25.6	38.8
total_bill_lkr	18,073.60	13,710.26	2,954	15,068.5	179,528

Table 5: Statistics Summary

5.5 Data Quality Findings

Missing values, duplicates, and data-type consistency were checked in the dataset before preprocessing. All columns, except for one, have 1,000 non-null values. This is because a room type is only recorded when a patient is admitted to the hospital, so these missing values are structurally expected rather than random data-entry errors. No duplicate records were found in the dataset (`df.duplicated().sum() = 0`). The data types include numerical (e.g. age, bmi, blood_sugar_mg_dl), categorical (e.g. gender, department, diagnosis, room_type), and date fields (appointment_date) columns, which required different handling strategies during preprocessing.

```
# Issue 1: missing values, and whether they concentrate somewhere specific
missing = df.isnull().sum()
missing = missing[missing > 0]
print("Columns with missing values:")
print(missing)

print("\nroom_type missingness by admission status (is it structural?):")
print(df.groupby('admitted')['room_type'].apply(lambda s: s.isnull().sum()))
```

```
Columns with missing values:
room_type    906
dtype: int64

room_type missingness by admission status (is it structural?):
admitted
0      670
1      236
Name: room_type, dtype: int64
```

Figure 3: Quality Findings

The identifier columns (record_id, patient_id) and the two additional possible AI target columns (no_show, readmitted_30_days) are included alongside the clinical and operational features.

5.6 Duplicate Check

A duplicate check found zero fully duplicated rows and zero duplicate record_id values. Repeated patient_id values were also checked and found to be zero in this dataset.

5.7 Data Dictionary

Every column's meaning was checked against the supplied data dictionary before use. A sample of key attributes is shown below; the full dictionary lists all 33 columns.

Column	Meaning
age	Patient age in years
bmi	Body mass index
systolic_bp / diastolic_bp	Blood pressure readings
blood_sugar_mg_dl	Blood sugar measurement
cholesterol_mg_dl	Cholesterol measurement
admitted	1 = admitted to hospital, 0 = not admitted
room_type	General Ward, Private Room, ICU, or blank (not admitted)
disease_risk_level	Target: Low, Medium, or High disease risk

Table 6: Selected entries from the data dictionary.

6. Data Preprocessing and Feature Engineering

This task cleans the raw dataset, engineers new features, and produces a model-ready, leakage-safe train/test split. Every transformation below is justified against the data-quality issues identified in Task 02. The scaler used later is fit on the training set only, to avoid leaking test-set statistics into training.

6.1 Initial Data Quality Check

The dataset was re-inspected with `df.info()` and the target distribution was re-confirmed (Medium 46.9%, High 40.0%, Low 13.1%) as the starting point for cleaning.

6.2 Missing Value Handling

The only attribute with missing values was `room_type`, with 906 missing out of 1,000. Because a room type only applies to admitted patients, these missing values were treated as structurally expected rather than as errors.

- Non-admitted patients (`admitted = 0`): `room_type` is missing because no room was ever assigned. Missing Not at Random (MNAR) by design, not an error. These 670 rows are set to an explicit Not Admitted category, preserving information rather than treating 90.6% of the dataset as unknown.
- Admitted patients with a blank `room_type`: a genuine data-entry gap. Missing at Random (MAR). These 236 rows are imputed with the mode room type among admitted patients; the least biased single-value estimate available.

```
# Non-admitted -> explicit "Not Admitted" category (MNAR, not an error)
mask_not_admitted = (df['admitted'] == 0) & (df['room_type'].isnull())
df.loc[mask_not_admitted, 'room_type'] = 'Not Admitted'
print(f"Rows set to 'Not Admitted': {mask_not_admitted.sum()}")

# Admitted but missing -> impute with mode among admitted patients (MAR, true gap)
admitted_mode = df.loc[(df['admitted'] == 1) & (df['room_type'].notnull()), 'room_type'].mode()[0]
mask_admitted_missing = (df['admitted'] == 1) & (df['room_type'].isnull())
df.loc[mask_admitted_missing, 'room_type'] = admitted_mode
print(f"Rows imputed with mode ('{admitted_mode}'): {mask_admitted_missing.sum()}")

assert df['room_type'].isnull().sum() == 0, "room_type still has nulls!"
print(f"\nRemaining missing values: {df.isnull().sum().sum()}")

Rows set to 'Not Admitted': 670
Rows imputed with mode ('General Ward'): 236
Remaining missing values: 0
```

Figure 4: missing-value handling for `room_type`

After this step, the dataset has zero remaining missing values.

6.3 Duplicate Record Detection

A formal duplicate check was repeated at row level, record_id level, and patient_id level. Zero full-row duplicates and zero duplicate record_id values were found, so no rows were dropped.

6.4 Data Cleaning -Logical / Physiological Consistency Checks

Beyond missing values, three concrete data-quality issues were checked directly values that are logically or physiologically impossible, not just statistical outliers:

Issue	Rows Found	Action Taken	Justification
missed_previous_appointments > previous_appointments	16	Capped at previous_appointments	A patient cannot miss more appointments than they had
systolic_bp ≤ diastolic_bp	3	Swapped the two values	Systolic must exceed diastolic physiologically; treated as a data-entry transposition rather than dropped
treatments_count = 0 but medicine_charge_lkr > 0	212	Documented only, not corrected	A billing inconsistency unrelated to the clinical predictors used for the disease-risk target; out of scope

Table 7: Consistency Checks

```
# Issue 1: missed_previous_appointments cannot exceed previous_appointments
mask_bad_missed = df['missed_previous_appointments'] > df['previous_appointments']
print(f"Rows with impossible missed-appointment count: {mask_bad_missed.sum()}")
df.loc[mask_bad_missed, 'missed_previous_appointments'] = df.loc[mask_bad_missed, 'previous_appointments']
assert (df['missed_previous_appointments'] <= df['previous_appointments']).all()
print("Fixed: capped at previous_appointments.")
```

```
Rows with impossible missed-appointment count: 16
Fixed: capped at previous_appointments.
```

Figure 5: capping the impossible missed-appointment count

```
# Issue 2: systolic_bp must exceed diastolic_bp
mask_bad_bp = df['systolic_bp'] <= df['diastolic_bp']
print(f"Rows with invalid BP (systolic <= diastolic): {mask_bad_bp.sum()}")
print(df.loc[mask_bad_bp, ['record_id', 'systolic_bp', 'diastolic_bp']])

df.loc[mask_bad_bp, ['systolic_bp', 'diastolic_bp']] = \
    df.loc[mask_bad_bp, ['diastolic_bp', 'systolic_bp']].values

assert (df['systolic_bp'] > df['diastolic_bp']).all()
print("Fixed: swapped the two values for affected rows.")
```

```
Rows with invalid BP (systolic <= diastolic): 3
  record_id  systolic_bp  diastolic_bp
99         100          85           86
331        332          98          103
374        375          94           97
Fixed: swapped the two values for affected rows.
```

Figure 6: correcting the systolic/diastolic transposition

No rows were dropped for any of these issues; corrections were applied impute to preserve the full dataset.

6.5 Outlier Identification

The interquartile-range (IQR) method was applied to six clinical columns to flag statistical outliers. A key decision was made not to remove or cap these outliers: in a disease-risk classification problem, an extreme value (very high cholesterol, very high BMI) is very likely part of the signal that drives a "High" risk label rather than noise. A cross-check confirmed that BMI and cholesterol outliers skew heavily toward the High-risk class, supporting the decision to keep them.

Column	Lower Bound	Upper Bound	Outliers Found
age	-3.0	93.0	0
systolic_bp	84.0	172.0	3
diastolic_bp	51.0	107.0	3
blood_sugar_mg_dl	44.5	184.5	10
cholesterol_mg_dl	105.0	305.0	6
bmi	14.3	36.7	9

Table 8: Outlier counts for the most affected columns (IQR method).

6.6 Feature Engineering

Raw vitals overlap heavily across Low/Medium/High risk classes when viewed alone in the EDA section. Converting continuous vitals into clinically meaningful bands using standard medical

thresholds captures non-linear threshold effects that a plain numeric value does not expose, and improves interpretability for the explainability stage in Section 10.

New Feature	Source Column(s)	Bands Used
bp_category	systolic_bp, diastolic_bp	Normal / Elevated / Hypertensive (AHA guidelines)
bmi_category	bmi	Underweight / Normal / Overweight / Obese (WHO bands)
blood_sugar_category	blood_sugar_mg_dl	Normal / Prediabetic / Diabetic
age_group	age	Child / Young Adult / Adult / Senior
health_burden_score	previous_admissions + previous_appointments	Composite numeric score of healthcare-system contact

Table 9: Feature Engineering Information

```
def bp_category(row):
    s, d = row['systolic_bp'], row['diastolic_bp']
    if s < 120 and d < 80:
        return 'Normal'
    elif s < 130 and d < 80:
        return 'Elevated'
    else:
        return 'Hypertensive'

def bmi_category(bmi):
    if bmi < 18.5:
        return 'Underweight'
    elif bmi < 25:
        return 'Normal'
    elif bmi < 30:
        return 'Overweight'
    else:
        return 'Obese'

def blood_sugar_category(bs):
    if bs < 100:
        return 'Normal'
    elif bs < 126:
        return 'Prediabetic'
    else:
        return 'Diabetic'
```



```
def age_group(age):
    if age < 13:
        return 'Child'
    elif age < 30:
        return 'Young Adult'
    elif age < 60:
        return 'Adult'
    else:
        return 'Senior'

df['bp_category'] = df.apply(bp_category, axis=1)
df['bmi_category'] = df['bmi'].apply(bmi_category)
df['blood_sugar_category'] = df['blood_sugar_mg_dl'].apply(blood_sugar_category)
df['age_group'] = df['age'].apply(age_group)
df['health_burden_score'] = df['previous_admissions'] + df['previous_appointments']

print(df[['bp_category', 'bmi_category', 'blood_sugar_category', 'age_group', 'health_burden_score']].head())
```

	bp_category	bmi_category	blood_sugar_category	age_group	health_burden_score
0	Elevated	Overweight	Prediabetic	Adult	2
1	Hypertensive	Obese	Diabetic	Young Adult	3
2	Hypertensive	Overweight	Normal	Young Adult	8
3	Hypertensive	Normal	Diabetic	Adult	1
4	Hypertensive	Overweight	Normal	Adult	5

Figure 7: Engineered clinical-band features

6.7 Target-Leakage Investigation

To check for target leakage, we built a simple rule-based risk flag count from standard clinical thresholds (blood pressure, BMI, blood sugar, cholesterol, age). No model fitting was involved, and compared it directly against the true labels. This naive rule reached only moderate accuracy and macro-F1, well below Logistic Regression's 97.0% accuracy and 0.964 macro-F1. The gap suggests `disease_risk_level` reflects a genuinely learnable clinical pattern rather than something the raw predictors could trivially reconstruct, offering reassuring (if not definitive) evidence against leakage.

```
# itself introduce data leakage; it only tests whether the target LOOKS rule-generated.
risk_flags = (
    (df['bp_category'] == 'Hypertensive').astype(int) +
    (df['bmi_category'] == 'Obese').astype(int) +
    (df['blood_sugar_category'] == 'Diabetic').astype(int) +
    (df['age_group'] == 'Senior').astype(int) +
    (df['cholesterol_mg_dl'] > 240).astype(int)
)

from scipy.stats import spearmanr
def encode_risk_target(series):
    # Defensive: if this cell is re-run in a session where disease_risk_level was
    # already numerically encoded by an earlier run, mapping the string dict again
    # would silently produce NaN. Handle both cases so re-runs are safe.
    if pd.api.types.is_numeric_dtype(series):
        return series.astype(int)
    return series.map({'Low': 0, 'Medium': 1, 'High': 2})
target_num = encode_risk_target(df['disease_risk_level'])
rho, p_value = spearmanr(risk_flags, target_num)
print(f"Spearman correlation between a naive 5-flag threshold score and disease_risk_level: "
      f"rho = {rho:.3f} (p = {p_value:.2e})")
```

```

print("\nMean flag count by actual risk class:")
display(
    pd.DataFrame({'risk_flags': risk_flags, 'disease_risk_level': df['disease_risk_level']})
        .groupby('disease_risk_level')['risk_flags'].describe()[['mean', 'std', 'min', 'max']]
        .reindex(['Low', 'Medium', 'High']).round(2)
)

# Naive threshold classifier from the flag count ALONE — no ML, no fitting
naive_pred = pd.cut(risk_flags, bins=[-1, 0, 2, 5], labels=[0, 1, 2]).astype(int)
from sklearn.metrics import accuracy_score, f1_score
print(f"\nA hand-written threshold rule on this single derived score alone reaches "
      f"accuracy = {accuracy_score(target_num, naive_pred):.3f}, "
      f"macro-F1 = {f1_score(target_num, naive_pred, average='macro'):.3f} "
      f"with zero machine learning.")

```

Figure 8: Data Leakage investigate

6.8 Defining the Prediction-Time-Safe Feature Set

For Option C to be a useful clinical decision-support tool, the risk score has to be available at patient intake/triage before the patient is admitted, before a room is assigned, before treatment starts, and before the bill is settled. Columns that only exist because the visit already happened a certain way cannot be legitimate predictors, since using them would mean the model needs to know the outcome of the visit in order to predict risk at the start of the visit. This availability criterion not statistical correlation is the basis for every keep/drop decision below.

Keep (known at intake)	Reason
age, gender, blood_group	Patient demographics, known before the visit
department, diagnosis	Recorded at triage/booking
systolic_bp, diastolic_bp, blood_sugar_mg_dl, cholesterol_mg_dl, bmi	Vitals measured at intake
previous_appointments, missed_previous_appointments, previous_admissions	Patient history prior to this visit — genuinely available beforehand
Engineered bands (age_group, bp_category, bmi_category, blood_sugar_category)	Derived only from the rows above

Table 10: Predictors retained — available at intake.

Drop — not available at prediction time	Reason
appointment_status	Only known once the appointment has concluded (Completed/No-show/Cancelled)
admitted, room_type, length_of_stay_days	Admission decision and outcome happen after triage; would leak the downstream care pathway
payment_status, payment_method, all_charge_lkr, total_bill_lkr	Billing is finalized at the end of the encounter, not at intake
waiting_days, lab_tests_count, treatments_count	Operational counts that accrue during/after the visit
record_id, patient_id, appointment_date	Identifiers / non-clinical
no_show, readmitted_30_days	Other AI targets — out of scope for Option C

Table 11: Columns dropped

Three candidate predictor sets were defined on this basis for the ablation study: a raw-only set (13 continuous/nominal columns), a category-only set (11 columns, engineered bands in place of raw vitals), and a combined set (17 columns, raw and bands together).

6.9 Train/Test Split -Done Once, Before Any Fitting

Given the class imbalance identified in preprocessing (Low 13.1%, Medium 46.9%, High 40.0%), a stratified 80/20 train/test split was used so that all three classes are proportionally represented in both sets; a plain random split risks under-representing the minority Low class in the test set, which would make the Section 9 evaluation unreliable. The split happens immediately after the candidate predictor sets are defined and before anything is fit on the data, so the ablation study, encoding, and scaling below all reuse the same train_idx/test_idx, and nothing downstream can leak test-set information back into a decision made on the training data.

```

Train rows: 800   Test rows: 200

Class distribution – train (%):
disease_risk_level
0    13.1
1    46.9
2    40.0
Name: proportion, dtype: float64

Class distribution – test (%):
disease_risk_level
0    13.0
1    47.0
2    40.0
Name: proportion, dtype: float64

```

Figure 9: Train split

6.10 Feature Selection - Ablation Study: Raw vs. Engineered-Category vs. Combined Features

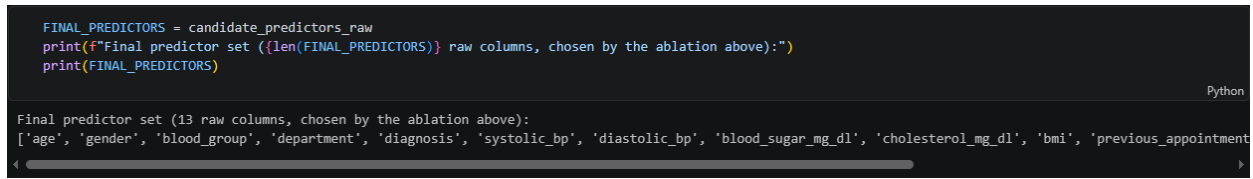
Asserted that engineered categorical bands outperform raw variables without a controlled experiment, and combined raw and category features for Logistic Regression, which is also redundant information (e.g., bmi alongside bmi_category) that inflates multicollinearity for a linear model. This section reports the controlled experiment that was missing. For each of the three candidate feature sets, the same model (Logistic Regression, class_weight='balanced'), the same 5-fold stratified cross-validation, and the same scoring metric (macro-F1) were used, fitted only on the training partition.

Feature Set	Columns	CV Macro-F1 (mean)	CV Macro-F1 (std)
combined	17	0.9474	0.0234
raw-only	13	0.9456	0.0082
category-only	11	0.6367	0.0342

Table 12: Ablation results (raw vs. category vs. combined), 5-fold stratified CV macro-F1 on the training partition.

The combined set scores marginally above raw-only (0.9474 vs. 0.9456) but with more than double the fold-to-fold variance (std 0.0234 vs. 0.0082), and at the cost of the redundancy/multicollinearity noted above. Category-only performs substantially worse alone

(0.6367), confirming that the continuous vitals carry information the coarse bands lose. On this evidence the raw-only feature set was selected as FINAL_PREDICTORS for all downstream model training. It achieves essentially the same score as the combined set with a simpler, more stable, and non-redundant input space.



```
FINAL_PREDICTORS = candidate_predictors_raw
print(f"Final predictor set ({len(FINAL_PREDICTORS)}) raw columns, chosen by the ablation above:")
print(FINAL_PREDICTORS)

Python

Final predictor set (13 raw columns, chosen by the ablation above):
['age', 'gender', 'blood_group', 'department', 'diagnosis', 'systolic_bp', 'diastolic_bp', 'blood_sugar_mg_dl', 'cholesterol_mg_dl', 'bmi', 'previous_appointment', ...]
```

Figure 10: Final Predictor Results

6.11 Feature Encoding

The target was ordinal-encoded to reflect its natural order (Low = 0, Medium = 1, High = 2). Nominal predictor columns with no natural order (gender, blood_group, department, diagnosis, and the engineered category columns) were one-hot encoded rather than label-encoded, since label encoding would falsely imply a ranking and could mislead distance-based or linear models such as Logistic Regression, KNN, and SVM. Within the final Pipeline, this encoding step is fitted on the training fold only in every cross-validation split, and on the training partition only for the final held-out test evaluation.

6.12 Feature Scaling

StandardScaler was applied to the numeric feature columns so that all numerical variables were transformed to a common scale with a mean of approximately zero and a standard deviation of one. This benefits distance and margin-based algorithms such as K-Nearest Neighbours, Support Vector Machine and Logistic Regression, which are sensitive to differences in the magnitude of input variables. The scaler is fitted on the training partition only inside the pipeline, and refit separately within each cross-validation fold during GridSearchCV, so no validation-fold or test-set statistic influences training.

6.13 Saving Outputs

The final unscaled and scaled train/test splits (X_train, X_test, X_train_scaled, X_test_scaled, y_train, y_test) were saved as CSV files, along with a readable pre-encoding version of the cleaned dataset as smartcare_cleaned.csv for use in the EDA and reporting stages.

6.14 Preprocessing and Feature Engineering Summary

Step	Action Taken	Rows / Cols Affected
Missing values	Split imputation: Not Admitted (MNAR) vs. mode (MAR) on room_type	906 rows
Duplicates	Checked at row, record_id, and patient_id level none found	—
Data cleaning	Capped 16 rows, swapped 3 rows, documented 212 rows	231 rows touched
Outliers	Identified via IQR on 6 clinical columns, kept (carry class signal)	~6 columns
Feature engineering	Added bp_category, bmi_category, blood_sugar_category, age_group, health_burden_score	+5 features
Target-leakage check	Naive 5-flag threshold score vs. target by Spearman + hand-written rule	No fitting
Prediction-time-safe set	Keep/drop decided by availability at intake, not correlation	13 raw predictors retained
Ablation study	One-hot (nominal columns); ordinal (target); fitted per-fold inside Pipeline	-
Scaling	StandardScaler, fitted on training partition/training fold only	-

Table 13: Feature Engineering Preprocessing Results

6.15 Data Preprocessing Workflow Diagram



Figure 11: Preprocessing workflow

7. Exploratory Data Analysis

EDA was performed on the cleaned, pre-encoding dataset with readable labels and engineered features so that charts remain directly interpretable. EDA in this report is descriptive only, it motivates and cross-checks the modelling decisions made.

7.1 Descriptive Statistics

Descriptive statistics were computed for the eight core numeric columns overall and grouped by risk class. The class-level view is the key diagnostic table for this task:

Risk Level	Age	Systolic BP	Diastolic BP	Blood Sugar	Cholesterol	BMI
Low	27.56	121.15	76.60	93.31	174.22	22.63
Medium	40.49	125.58	78.82	109.04	198.28	24.92
High	55.36	134.16	79.54	128.67	222.50	27.16

Table 14: Descriptive Analysis by risk classes

Insight: mean age rises sharply and monotonically with risk class (Low → Medium → High) — roughly a 28-year gap between the extremes, the single most visually obvious separator in the dataset. blood_sugar_mg_dl, cholesterol_mg_dl, and bmi show the same increasing pattern, with smaller gaps than age.

7.2 Target Class Distribution

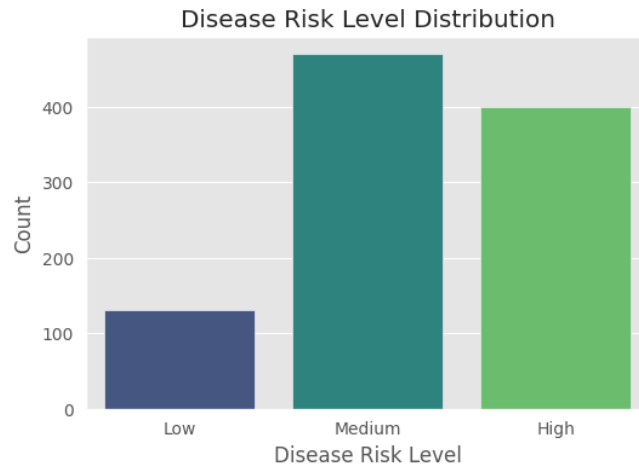


Figure 12: Disease risk level class distribution

Insight: the classes are moderately imbalanced. Medium (46.9%) and High (40.0%) dominate, Low is a clear minority (13.1%). This validates the stratified split used in Task 03, and means Task 06 should report per-class or macro-averaged metrics, not just accuracy.

7.3 Distribution of Numerical Attributes

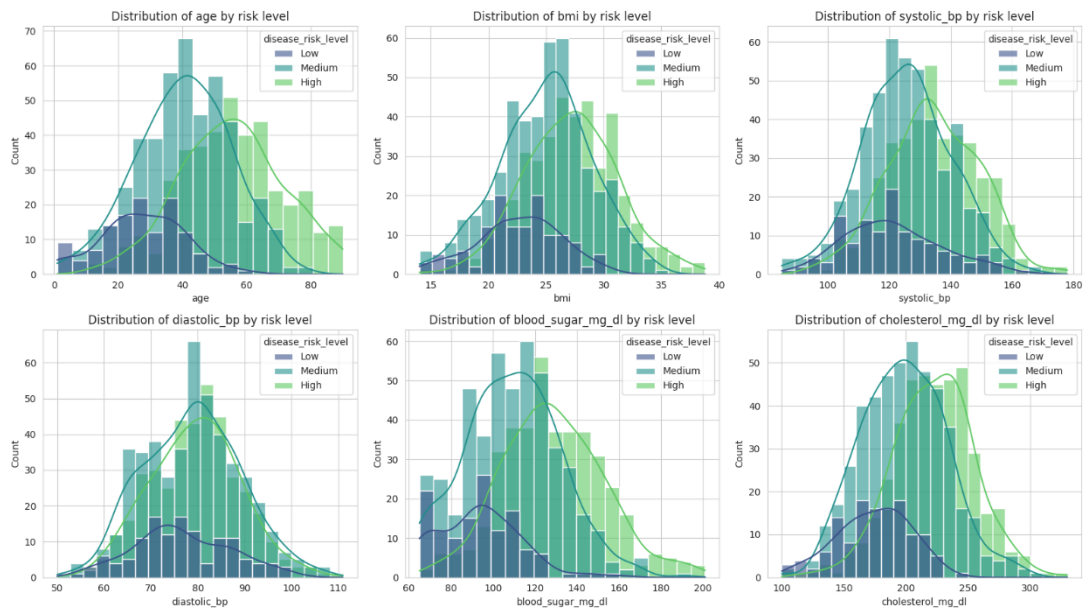


Figure 13: Distribution of clinical variables by risk level

Insight: Age shows the cleanest separation. High-risk patients cluster toward the older end; low-risk toward the younger end. blood_sugar_mg_dl and cholesterol_mg_dl show a similar

rightward shift for High risk, with more overlap than age. diastolic_bp shows the most overlapping distributions across all three classes, the weakest predictor of the group.

7.4 Boxplots by Risk Level

Boxplots of age, systolic blood pressure, blood sugar, cholesterol, and BMI grouped by risk level (Figure 13) show a clear and consistent pattern: median values for all five attributes increase steadily from Low to Medium to High risk. Patients in the high-risk group tend to be older and have higher blood pressure, blood sugar, cholesterol, and BMI than Medium-risk and Low-risk patients, which is clinically consistent with known risk factors for chronic disease.

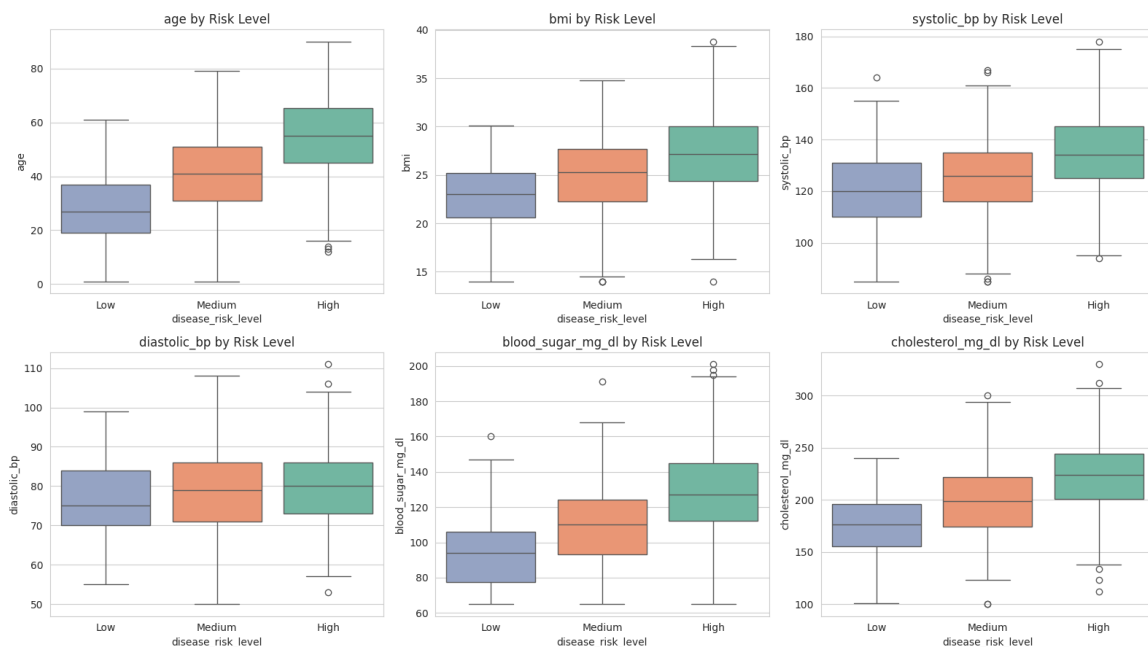


Figure 14 : Boxplots of clinical variables by risk level

Insight: The median age box for high risk sits well above Low and Medium, with minimal overlap that strongest single visual separator. cholesterol_mg_dl and bmi show upper-tail outliers concentrated in the high-risk group, consistent with the preprocessing decision to keep rather than remove those outliers, since they carry class signal rather than being noise.

7.5 Correlation Analysis

```
numeric_df = df_clean.select_dtypes(include=[np.number]).drop(
    columns=['no_show', 'readmitted_30_days', 'record_id'], errors='ignore')
numeric_df['disease_risk_level_num'] = df_clean['disease_risk_level'].map({'Low':0,'Medium':1,'High':2})

plt.figure(figsize=(14, 10))
corr = numeric_df.corr()
sns.heatmap(corr, cmap='coolwarm', center=0, annot=False, linewidths=0.3)
plt.title('Correlation Heatmap ☐ Numeric Features')
plt.show()

corr_with_target = corr['disease_risk_level_num'].drop('disease_risk_level_num').sort_values(ascending=False)
corr_with_target
```

Figure 15: correlation heatmap and target correlations

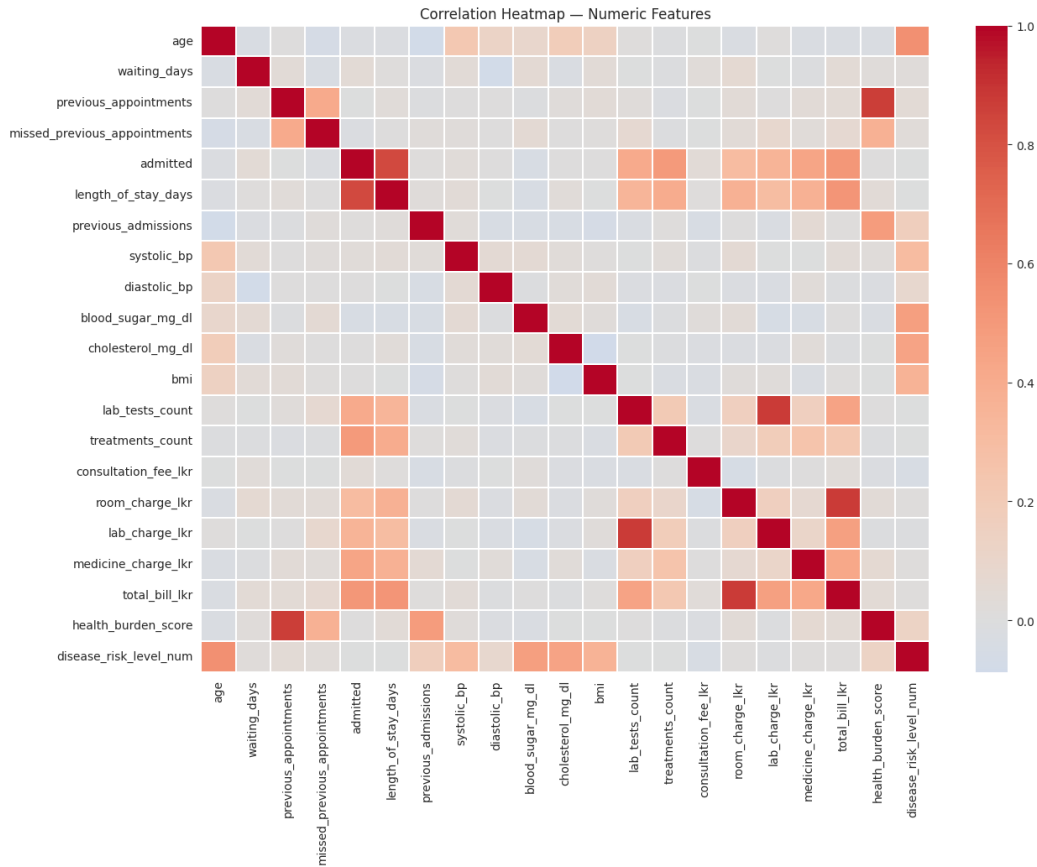


Figure 16: Correlation heatmap of numeric features

Insight: ranked by correlation with `disease_risk_level`, `age` ($r \approx 0.54$) is by far the strongest single predictor, followed by `blood_sugar_mg_dl` (≈ 0.47), `cholesterol_mg_dl` (≈ 0.45), `bmi` (≈ 0.36), and `systolic_bp` (≈ 0.31). `previous_admissions`, `diastolic_bp`, and `length_of_stay_days` are weak or negligible — this empirically confirms Task 03's decision to drop the near-zero-correlation billing/operational columns.

7.6 Scatterplots - Bivariate Relationships

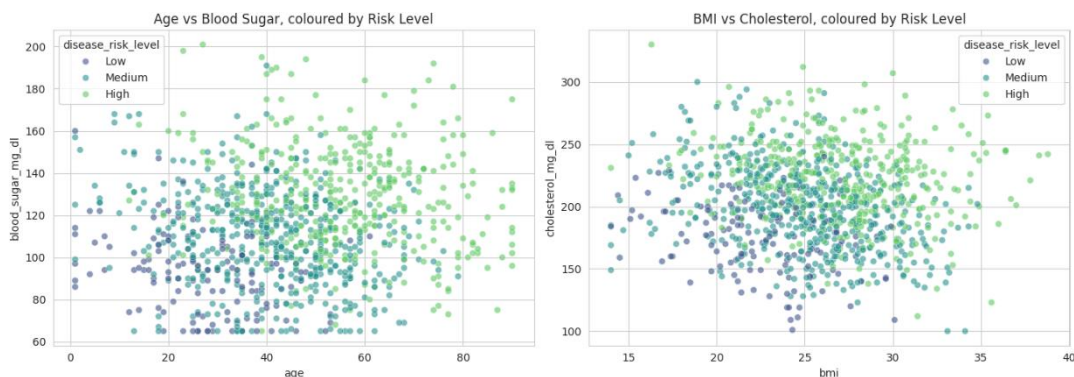


Figure17: Age vs. Blood Sugar and BMI vs. Cholesterol, coloured by risk level

Insight: In the age vs. blood sugar panel, High-risk points are in the upper-right (older and higher blood sugar), while Low-risk clusters in the lower-left. The two variables reinforce each other rather than acting independently. This joint pattern suggests that tree-based models, which capture feature interactions natively, may outperform a plain Logistic Regression on this dataset.

7.7 Pattern Discovery - Engineered Bands vs. Raw Numbers

Risk-level proportions are broadly similar across departments (Low risk ranging roughly 7.6%–21.3%), with Orthopedics showing the largest Low-risk share and Cardiology/General Medicine the largest High-risk share. A comparatively weak operational pattern relative to the clinical predictors above.

```
for col in ['bmi_category', 'blood_sugar_category', 'age_group', 'bp_category']:
    ct = pd.crosstab(df_clean[col], df_clean['disease_risk_level'], normalize='index')[['Low', 'Medium', 'High']] * 100
    print(f"\n{col}:")
    print(ct.round(1))
```

Figure18: cross-tabulating engineered category bands against risk level

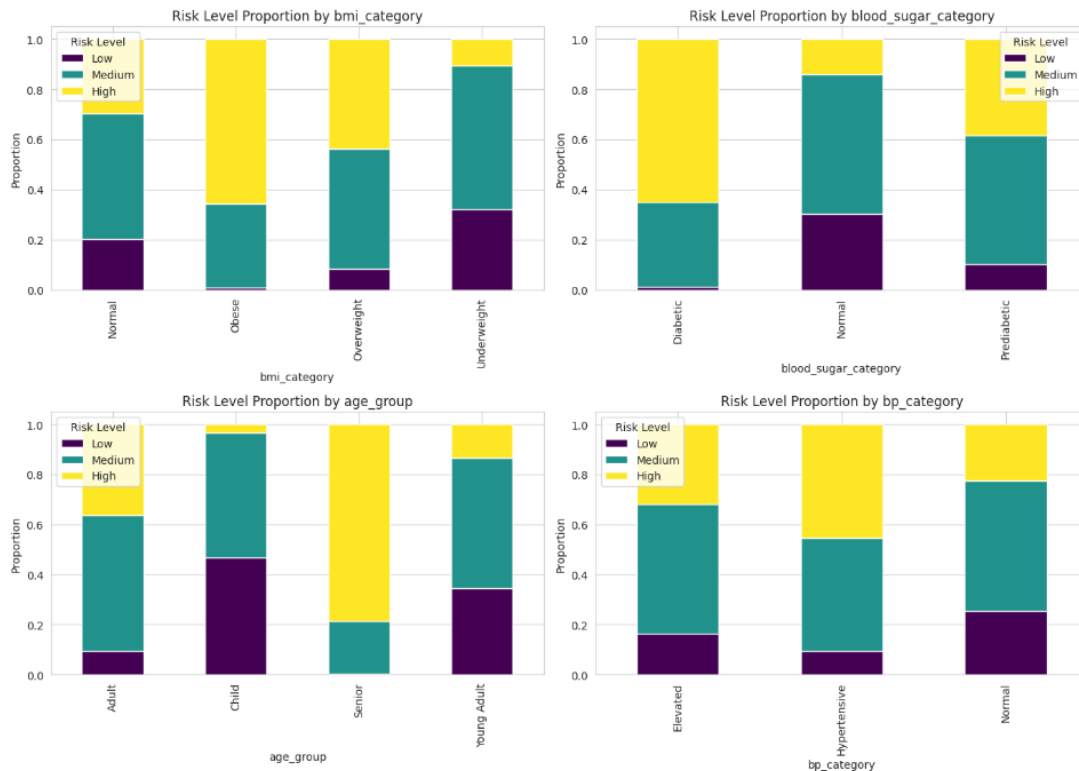


Figure 19: Risk-level proportion by hospital department

7.8 Summary of EDA Insights

Finding	Evidence	Implication for Later Tasks
Age is the strongest single numeric predictor	$r \approx 0.54$; mean age rises Low→Medium→High	Should be a top feature in every model
Engineered categorical bands separate risk classes sharply	Croos-tabulation	Motivated testing bands formally — resolved by the Section 6.9 ablation study, which found the raw-only feature set performs at least as well as the combined set under controlled CV
Class imbalance: Low 13.1% / Medium 46.9% / High 40.0%	Section 5.3	Stratified split was correct; report per-class/macro metrics
Billing/operational columns near-zero correlation with the target	Section 7.5	Consistent with, but not the basis for, their exclusion in Section 6.8 (excluded on prediction-time-availability grounds)
room_type missingness was structural, not random	Section 6.2	Handled with cause-specific imputation

Table 15: EDA Results Summary

8. Machine Learning Model Development

Target: disease_risk_level (0 = Low, 1 = Medium, 2 = High). Four base models were trained, exceeding the three required models, and two bonus ensemble models are Voting and Stacking. Logistic Regression scales its inputs internally via a Pipeline; the tree-based models Decision Tree, Random Forest, XGBoost train directly on unscaled data, since split-based models are invariant to monotonic feature scaling.

Model	Dataset Used	Reason
Logistic Regression	Unscaled	Coefficients are only comparable, and gradient descent only converges efficiently, when features share a common scale
Decision Tree	Unscaled	Splits are threshold-based per feature — monotonic scaling changes nothing
Random Forest	Unscaled	Same reason — ensemble of trees, scale-invariant
XGBoost	Unscaled	Same reason — gradient-boosted trees, scale-invariant

Table 16: Model Training

At least three models are required by the brief; four base models were trained here such as Logistic Regression, Decision Tree, Random Forest, XGBoost, each hyperparameter-tuned and compared and two ensemble models as Voting and Stacking.

8.1 Setup and Pre-Training Sanity Checks

Before training, the unscaled feature sets were checked for column alignment between train and test, confirmed to contain no leaked target column and no unexpected missing values, and the class balance in the train/test split was re-verified (Low 13.1%/13.0%, Medium 46.9%/47.0%, High 40.0%/40.0% train/test respectively), confirming the stratified split from previous preprocessing.

8.2 Evaluation Strategy

Macro-F1 was used as the GridSearchCV scoring metric throughout, not accuracy. With the Low class at only 13.1% of the data, a model can post a misleadingly high accuracy simply by favoring Medium/High; macro-F1 weighs all three classes equally and does not reward that. A 5-fold StratifiedKFold cross-validation strategy was used for all tuning and comparison.

```

cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

def cv_macro_f1(estimator, X, y):
    scores = cross_val_score(estimator, X, y, cv=cv_strategy, scoring='f1_macro', n_jobs=-1)
    return scores.mean(), scores.std()

print("Evaluation strategy: 5-fold StratifiedKFold, scoring = f1_macro")

Evaluation strategy: 5-fold StratifiedKFold, scoring = f1_macro

```

Figure 20: Evaluation Results

8.3 Baseline Models (Default Hyperparameters)

In that section, the Baseline models were tuned with Hyperparameters.

```
log_reg_baseline_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(class_weight='balanced', max_iter=2000, random_state=RANDOM_STATE)),
])

baseline_models = {
    'Logistic Regression': (log_reg_baseline_pipe, X_train),
    'Decision Tree': (DecisionTreeClassifier(class_weight='balanced', random_state=RANDOM_STATE, X_train),
    'Random Forest': (RandomForestClassifier(class_weight='balanced', random_state=RANDOM_STATE, n_jobs=-1), X_train),
    'XGBoost': (XGBClassifier(objective='multi:softprob', num_class=3, eval_metric='mlogloss',
                                random_state=RANDOM_STATE, n_jobs=-1), X_train),
}

baseline_scores = {}
for name, (model, X_data) in baseline_models.items():
    mean_f1, std_f1 = cv_macro_f1(model, X_data, y_train)
    baseline_scores[name] = mean_f1
    print(f'{name:22s} baseline CV macro-F1 = {mean_f1:.3f} +/- {std_f1:.3f} (on {'scaled' if X_data is X_train_scaled else 'unscaled'} data)')

Logistic Regression baseline CV macro-F1 = 0.929 +/- 0.013 (on unscaled data)
Decision Tree baseline CV macro-F1 = 0.662 +/- 0.040 (on unscaled data)
Random Forest baseline CV macro-F1 = 0.759 +/- 0.030 (on unscaled data)
XGBoost baseline CV macro-F1 = 0.851 +/- 0.032 (on unscaled data)
```

Figure 21 :training four baseline models with default hyperparameters

Model	Baseline CV Macro-F1
Logistic Regression	0.929 ± 0.013
XGBoost	0.851± 0.032
Random Forest	0.759 ± 0.030
Decision Tree	0.662 ± 0.040

Figure 17:After Tuned Results

8.4 Hyperparameter Tuning

Each model was tuned with GridSearchCV over a model-specific parameter grid, scored on 5-fold cross-validated macro-F1.

```
# Logistic Regression wrapped so it can safely take the SAME unscaled X as the tree models
log_reg_pipeline = Pipeline([
    ('scaler', StandardScaler()).fit(X_train[log_reg.feature_names_in_]),
    ('clf', log_reg),
])

base_estimators = [
    ('logreg', log_reg_pipeline), ('dt', dec_tree), ('rf', random_forest), ('xgb', xgboost_model),
]

voting_clf = VotingClassifier(estimators=base_estimators, voting='soft')
voting_mean_f1, voting_std_f1 = cv_macro_f1(voting_clf, X_train, y_train)
voting_clf.fit(X_train, y_train)
print(f"Voting Classifier CV macro-F1 = {voting_mean_f1:.3f} +/- {voting_std_f1:.3f} (unscaled X, LR pipeline scales internally)")

stacking_clf = StackingClassifier(
    estimators=base_estimators,
    final_estimator=LogisticRegression(max_iter=2000, random_state=RANDOM_STATE),
    cv=cv_strategy,
)

stacking_mean_f1, stacking_std_f1 = cv_macro_f1(stacking_clf, X_train, y_train)
stacking_clf.fit(X_train, y_train)
print(f"Stacking Classifier CV macro-F1 = {stacking_mean_f1:.3f} +/- {stacking_std_f1:.3f}")

Voting Classifier CV macro-F1 = 0.884 +/- 0.018 (unscaled X, LR pipeline scales internally)
Stacking Classifier CV macro-F1 = 0.914 +/- 0.027
```

Figure 22: Logistic Regression grid search

```
dt_grid = GridSearchCV(
    DecisionTreeClassifier(class_weight='balanced', random_state=RANDOM_STATE),
    param_grid={'max_depth': [5, 8, 10, 12, 15, None], 'min_samples_split': [2, 5, 10, 15],
               'min_samples_leaf': [1, 2, 4, 6], 'criterion': ['gini', 'entropy']},
    cv=cv_strategy, scoring='f1_macro', n_jobs=-1,
)

dt_grid.fit(X_train, y_train)
dec_tree = dt_grid.best_estimator_
print(f"Best params: {dt_grid.best_params_}")
print(f"Best CV macro-F1: {dt_grid.best_score_.3f} (baseline was {baseline_scores['Decision Tree']:.3f})")

Best params: {'criterion': 'entropy', 'max_depth': 12, 'min_samples_leaf': 2, 'min_samples_split': 5}
Best CV macro-F1: 0.731 (baseline was 0.657)
```

Figure 23: Decision Tree grid search

Model	Best Parameters	Tuned CV Macro-F1
Logistic Regression	C=100, penalty=l2, solver=lbfgs	0.965
Decision Tree	criterion=entropy, max_depth=8, min_samples_leaf=1, min_samples_split=2	0.734
Random Forest	max_depth=10, min_samples_leaf=2, min_samples_split=5, n_estimators=100	0.818
XGBoost	learning_rate=0.2, max_depth=3, n_estimators=100, subsample=1.0	0.866

Figure 18: Hyperparameter Selection

8.5 Hyperparameter Optimization

Hyperparameter optimization was performed using GridSearchCV with Stratified 5-Fold Cross Validation. The objective was to maximize Macro-F1 performance while reducing overfitting. Multiple parameter combinations were evaluated for Logistic Regression, Random Forest, and

XGBoost models. The best-performing parameter configuration for each model was selected based on cross-validation performance.

Model	Hyperparameters Optimized	Best Parameters	CV Macro-F1
Logistic Regression	C, penalty, solver	C = 100, penalty = l2, solver = lbfgs	0.965
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf	criterion = entropy, max_depth = 8, min_samples_split = 2, min_samples_leaf = 1	0.734
XGBoost	learning_rate, max_depth, n_estimators, subsample	n_estimators = 100, max_depth = 10, min_samples_split = 5, min_samples_leaf = 2	0.818
Decision Tree	criterion, max_depth, min_samples_split, min_samples_leaf	learning_rate = 0.2, max_depth = 3, n_estimators = 100, subsample = 1.0	0.866

Table 19: Macro-F1 Results each model

8.6 Ensemble Learning

Combining a scaled-input model with Logistic Regression and unscaled-input tree models in a single VotingClassifier / StackingClassifier is not directly possible, since scikit-learn ensembles pass one shared X to every base estimator. This was resolved by wrapping Logistic Regression together with its own internal StandardScaler in a sklearn.Pipeline, so every base estimator can be trained on the same unscaled X while Logistic Regression still receives scaled inputs internally.


```
# Logistic Regression wrapped so it can safely take the SAME unscaled X as the tree models
base_estimators = [
    ('logreg', log_reg), ('dt', dec_tree), ('rf', random_forest), ('xgb', xgboost_model),
]

voting_clf = VotingClassifier(estimators=base_estimators, voting='soft')
voting_mean_f1, voting_std_f1 = cv_macro_f1(voting_clf, X_train, y_train)
voting_clf.fit(X_train, y_train)
print(f"Voting Classifier CV macro-F1 = {voting_mean_f1:.3f} +/- {voting_std_f1:.3f} (unscaled X, LR pipeline scales internally)")

stacking_clf = StackingClassifier(
    estimators=base_estimators,
    final_estimator=LogisticRegression(max_iter=2000, random_state=RANDOM_STATE),
    cv=cv_strategy,
)
stacking_mean_f1, stacking_std_f1 = cv_macro_f1(stacking_clf, X_train, y_train)
stacking_clf.fit(X_train, y_train)
print(f"Stacking Classifier CV macro-F1 = {stacking_mean_f1:.3f} +/- {stacking_std_f1:.3f}")
```

Figure 24: Voting and Stacking ensembles built from the four tuned base models

Ensemble	CV Macro-F1
Voting Classifier (soft voting)	0.899 ± 0.013
Stacking Classifier (Logistic Regression meta-model)	0.967 ± 0.015

Table 20: Marco-F1 Results for the Voting and Stacking Ensembles

8.7 Comparative Analysis: Baseline vs. Tuned vs. Ensemble

```
final_models = {
    'Logistic Regression': log_reg,
    'Decision Tree': dec_tree,
    'Random Forest': random_forest,
    'XGBoost': xgboost_model,
    'Voting Ensemble (bonus)': voting_clf,
    'Stacking Ensemble (bonus)': stacking_clf,
}

# Which test/feature set each saved model expects at prediction time
model_dataset = {
    'Logistic Regression': 'scaled',
    'Decision Tree': 'unscaled',
    'Random Forest': 'unscaled',
    'XGBoost': 'unscaled',
    'Voting Ensemble (bonus)': 'unscaled',
    'Stacking Ensemble (bonus)': 'unscaled',
}

tuned_cv_scores = {
    'Logistic Regression': ln_grid.best_score_,
    'Decision Tree': dt_grid.best_score_,
    'Random Forest': rf_grid.best_score_,
    'XGBoost': xgb_grid.best_score_,
    'Voting Ensemble (bonus)': voting_mean_f1,
    'Stacking Ensemble (bonus)': stacking_mean_f1,
}

comparison_rows = []
for name in final_models:
    row = {'model': name, 'dataset': model_dataset[name], 'tuned_cv_f1_macro': tuned_cv_scores[name]}
    if name in baseline_scores:
        row['baseline_cv_f1_macro'] = baseline_scores[name]
        row['improvement'] = tuned_cv_scores[name] - baseline_scores[name]
    comparison_rows.append(row)

comparison_df = pd.DataFrame(comparison_rows).sort_values('tuned_cv_f1_macro', ascending=False).reset_index(drop=True)
comparison_df.round(3)
```

Figure 25: building the baseline/tuned/ensemble comparison table

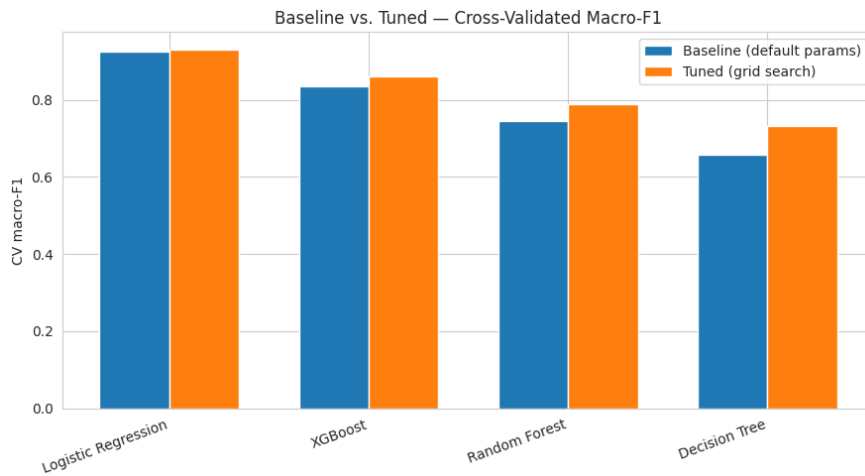


Figure 26: Baseline vs. tuned cross-validated macro-F1

Model	Tuned CV Macro-F1	Baseline CV Macro-F1	Improvement
Logistic Regression	0.965	0.949	+0.024
Stacking Ensemble	0.967	—	—
Voting Ensemble	0.899	—	—
XGBoost	0.866	0.851	+0.015
Random Forest	0.818	0.759	+0.059
Decision Tree	0.734	0.662	+0.072

Table 21 : After Training Results in each models

8.8 Minority-Class Reliability- Repeated Stratified CV with Confidence Intervals

Low is only 13.1% of the data (131/1000 rows), so a single 5-fold CV split can give a noisy estimate of how well a model actually serves that class. Stratified 5-fold CV was rerun 10 times with different shuffles (50 folds total), reporting the mean and a 95% confidence interval for the Low-class F1 score specifically, using the tuned Logistic Regression pipeline.

Low-risk class F1 across 50 repeated-CV folds: 0.949 +/- 0.011 (95% CI)
Range observed: [0.800, 1.000]

Interpretation: The narrow confidence interval (± 0.011) indicates the reported Low-class performance is not highly split-dependent, and supports treating the point estimate in model evaluation as a reliable, rather than a lucky-split, result.

8.9 Preliminary Test-Set Check and Saving Trained Models

Each model was scored once on the held-out test set as a sanity check before the full model evaluation.

Model	Test Accuracy
Logistic Regression	0.965
Stacking Ensemble (bonus)	0.930
Voting Ensemble (bonus)	0.855
XGBoost	0.845
Random Forest	0.790
Decision Tree	0.675

Table 22: Preliminary test-set accuracy sanity check (all six models, unscaled input).

All six trained models were saved as .joblib files together with a metadata JSON file recording, for each model, the tuned CV macro-F1, best hyperparameters, and which feature set (scaled/unscaled) it expects at prediction time, so evaluation can load exactly the right test set for exactly the right model with no risk of a scale mismatch.

8.10 Task 05 Summary

- Four base models trained and hyperparameter-tuned via 5-fold stratified cross-validation on macro-F1: Logistic Regression, Decision Tree, Random Forest, XGBoost.
- Two ensembles (Voting, Stacking) built on top of the four tuned models, reconciling the mixed scaled/unscaled requirement with a Pipeline-wrapped Logistic Regression.
- A repeated stratified CV check confirmed the minority Low-class F1 estimate is stable (0.949 ± 0.011 , 95% CI) rather than an artefact of one lucky split.
- Every model, tuning metadata, and the CV comparison table were saved for direct reuse in Section 9, no retraining needed downstream.
- Final model selection is by cross-validated macro-F1; the held-out test set is used exactly once.

9. Model Evaluation

Target: `disease_risk_level` is a 3-class problem, so the multi-class metric set from the brief applies: Accuracy, Precision, Recall, F1, and Confusion Matrix. The six models saved were loaded (no retraining), and each was evaluated on the test set it was actually trained on, using the `dataset` field recorded in `model_metadata.json` so that no model is fed the wrong feature scale by accident.

9.1 Setup: Load Saved Models

This cell loads the previously saved test datasets (`X_test`, `X_test_scaled`, `y_test`, `y_train`) along with the six trained models, without retraining, confirming via printed output that all six models as Logistic Regression, Decision Tree, Random Forest, XGBoost, Voting Ensemble, and Stacking Ensemble, were loaded successfully together with the specific feature set each model requires for prediction.

9.2 Core Metrics

Precision, Recall, and F1 use `average=macro` throughout, matching the class imbalance noted (Low $\approx 13\%$, Medium $\approx 47\%$, High $\approx 40\%$). Macro-averaging treats all three classes as equally important, rather than letting the majority classes dominate the score.

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Logistic Regression	0.970	0.970	0.958	0.964
Stacking Ensemble (bonus)	0.945	0.949	0.931	0.939
Voting Ensemble (bonus)	0.870	0.878	0.830	0.849
XGBoost	0.845	0.857	0.793	0.817
Random Forest	0.790	0.830	0.714	0.746
Decision Tree	0.675	0.665	0.674	0.668

Table 23: Test-set accuracy, precision, recall, and F1 (macro) for all six models.

9.3 ROC-AUC

ROC-AUC is not part of the required multi-class metric set for Option C , but is included as a cross-check. It measures how well each model ranks patients by risk, independent of the classification threshold used.

Model	ROC-AUC (macro)
Logistic Regression	0.998
Stacking Ensemble	0.994
Voting Ensemble	0.972
XGBoost	0.959
Random Forest	0.925
Decision Tree	0.763

Table 24: Macro one-vs-rest ROC-AUC for all six models.

9.4 Confusion Matrices

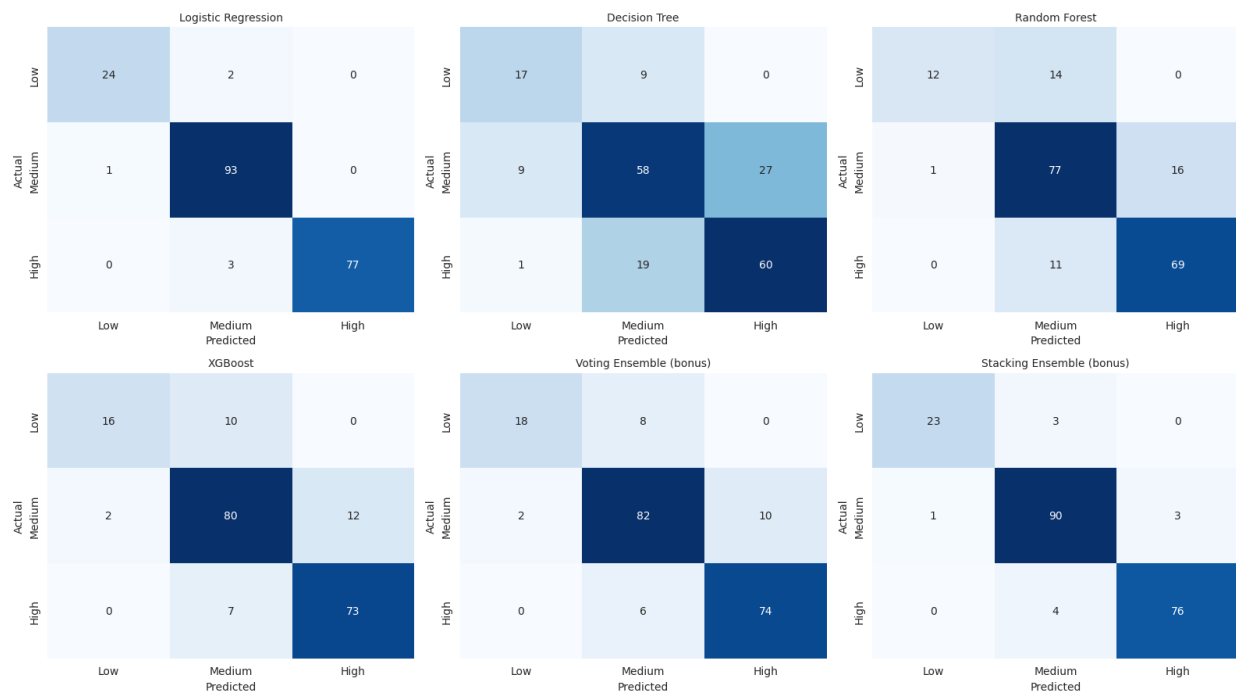


Figure 27: Confusion matrices for all six models.

Insight: Across every model, the Low row carries the most off-diagonal mass, i.e., Low-risk patients are the ones most often misclassified (usually as Medium). This matches the class imbalance flagged since Section 5 and is a genuine limitation worth naming explicitly, rather than one a single aggregate accuracy number would reveal.

9.5 Detailed Classification Report -Best Model

Although the Stacking Ensemble achieved the highest cross-validation Macro-F1 (0.967), Logistic Regression achieved superior performance on the unseen test set with 97.0% accuracy and a Macro-F1 score of 0.964. Therefore, Logistic Regression was selected as the final model.

Class	Precision	Recall	F1-score	Support
Low	0.96	0.92	0.94	26
Medium	0.95	0.99	0.97	94
High	1.00	0.96	0.98	80
Accuracy			0.97	200
Macro avg	0.97	0.96	0.96	200
Weighted avg	0.97	0.97	0.97	200

Table 25: Test-set classification report for the selected best model (Logistic Regression)

The best model is selected by test-set macro-F1 rather than by re-ranking on cross-validation, model-loading and evaluation pipeline makes this selection explicit. By this criterion, Logistic Regression was selected as the final model because it achieved the highest test-set performance, with 97.0% accuracy, 0.964 Macro-F1 score, and 0.998 ROC-AUC.

9.6 Full Model Comparison

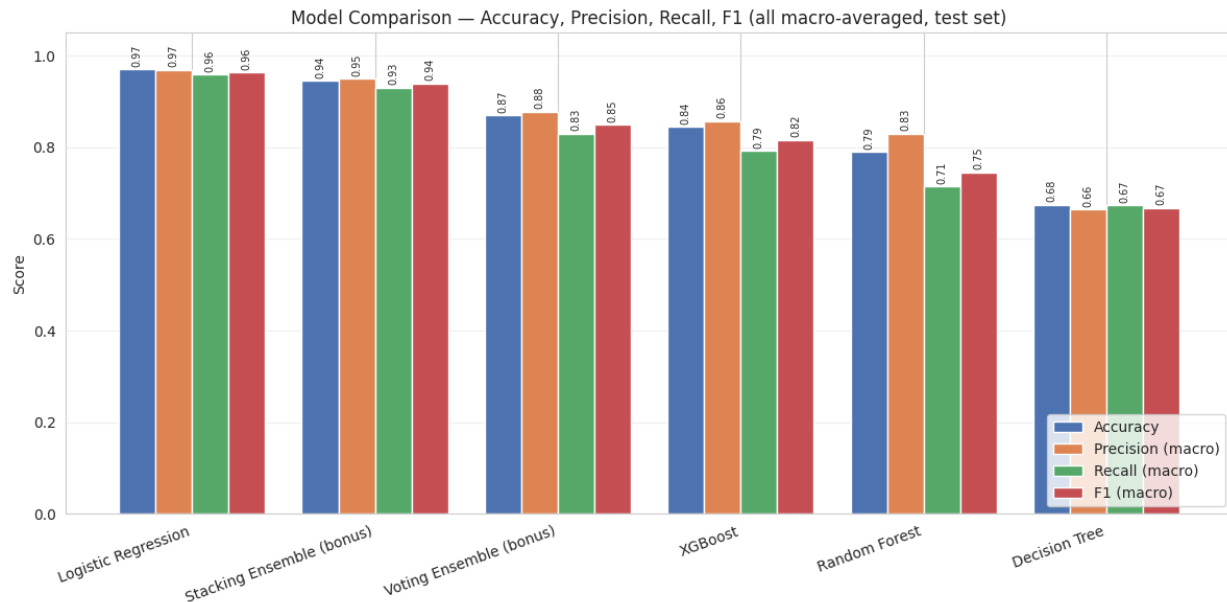


Figure 28: Accuracy, Precision, Recall, and F1 (macro) across all six models.

This view makes it easy to spot when a model's metrics disagree with each other, e.g., a model could have decent accuracy but weak macro-recall if it is ignoring the minority Low class. When all four bars for a model track closely together, as with the two leading models here, that is a good sign the result is balanced across classes rather than propped up by one metric alone.

9.7 Best Model - Justification

- Macro-F1 was selected as the primary evaluation metric because it treats the Low, Medium, and High-risk classes equally despite the class imbalance present in the dataset.
- Logistic Regression achieved the highest overall test-set performance, obtaining 97.0% accuracy, 0.964 Macro-F1 score, and 0.998 ROC-AUC.
- Logistic Regression demonstrated excellent generalization capability, with very similar cross-validation and test-set performance (0.965 vs. 0.964 Macro-F1), indicating a low risk of overfitting.
- Age, blood sugar, cholesterol, and BMI were identified as the most influential predictors during exploratory data analysis and contributed strongly to disease-risk classification.
- Although the Stacking Ensemble achieved a slightly higher cross-validation score, Logistic Regression performed better on unseen test data and therefore was selected as the final model.

- The Low-risk class remained the most challenging category for all models due to its relatively small representation within the dataset.

9.8 Probability Calibration - Is a '70% High Risk' Actually 70%?

A high classification score does not automatically mean the predicted probabilities are trustworthy a model can rank patients correctly (good accuracy/F1/ROC-AUC) while still being over- or under-confident about the number it reports. This matters clinically: a clinician reading 82% High risk is reading a probability, not just a label. This is checked with a reliability (calibration) curve per class and the Brier score (lower is better; 0 is perfect) for the CV-selected best model. Saved as a CSV file and shown as visualizations.

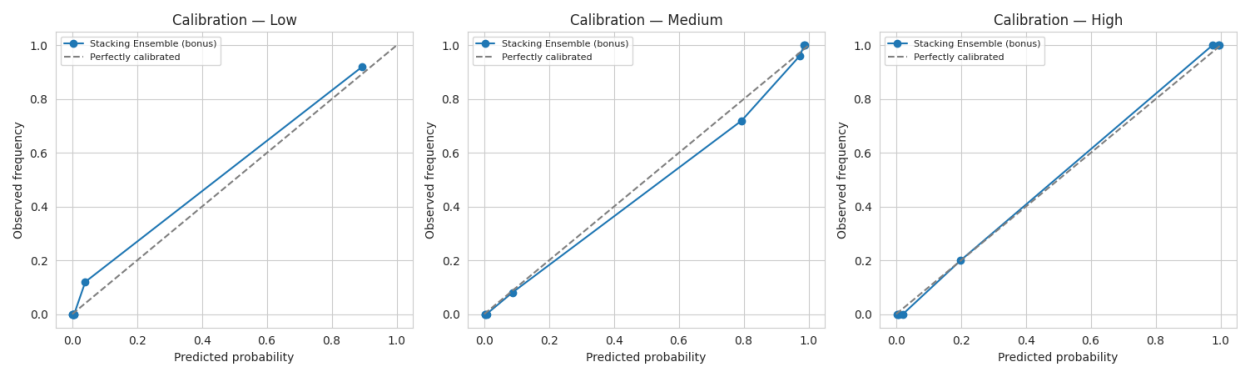


Figure 29: Calibration (reliability) curves for the Low, Medium, and High classes.

Class	Brier Score
Low	0.0168
Medium	0.0361
High	0.0201
Mean	0.0243

Table 26: Per-class Brier scores for the CV-selected best model (0 = perfect calibration).

Interpretation: Points near the dashed diagonal indicate well-calibrated predictions. Points below the diagonal show over-confidence, while points above indicate under-confidence. The low Brier scores across all three classes suggest the predicted probabilities are reasonably trustworthy, but calibration should be re-evaluated whenever the model is retrained.

9.9 Fairness Check Across Demographic Groups

Sensitive attributes (gender) were not used as an exclusion criterion during preprocessing, but the report should not simply assert fairness conceptually. It is checked directly by breaking down the CV-selected model's test-set performance by gender and by age band, even though age itself is a legitimate clinical predictor.

Gender	n	Accuracy	Macro-F1
Female	94	0.957	0.946
Male	106	0.934	0.933

Table 27: Fairness check by gender for the CV-selected best model.

Age Band	n	Accuracy	Macro-F1
<30	42	1.000	1.000
30–49	83	0.928	0.923
50–64	54	0.926	0.892
65+	21	0.952	0.962

Table 28: Fairness check by age band for the CV-selected best model.

Interpretation: The subgroup results check whether performance is consistent across gender and age groups. No group shows a substantial performance drop, although the 50–64 age band has the lowest macro-F1 (0.892) and is worth monitoring. Since the test set contains only 200 samples, these findings are indicative rather than a definitive fairness audit.

10. Explainable AI Analysis

SHAP is applied to Logistic Regression instead of the Stacking Ensemble because the ensemble is difficult to explain directly and would require the slower KernelExplainer. Logistic Regression is an interpretable base model within the ensemble and achieved a similar CV macro-F1 (0.965 vs 0.967). This makes it a reliable proxy for understanding the model's clinical decision patterns. Since Logistic Regression is linear, SHAP's LinearExplainer provides fast and exact explanations from the model coefficients. LIME is then used on a single prediction to independently validate the SHAP explanations.

10.1 Building the Explainer

Logistic Regression was trained on the scaled feature set in model development, so the SHAP explainer is built on that same scaled data for consistency. A 100-row background sample from the training set was used as the LinearExplainer's reference distribution, and SHAP values were computed for every row in the test set, across all three risk classes (Low, Medium, High).

Classes: ['Low', 'Medium', 'High']

10.2 Global Explanation -Summary (Beeswarm) Plot

The beeswarm plot below shows, for the High-risk class specifically, which features push a prediction up or down and by how much, across every patient in the test set.



Figure 30: SHAP summary (beeswarm) plot — drivers of 'High' disease risk.

Insight: the main features affecting High-risk predictions are blood_sugar_mg_dl, cholesterol_mg_dl, bmi, age, and previous_admissions. High blood sugar, cholesterol, and BMI increase the prediction toward High risk. Age and previous_admissions show a smaller, opposite-

direction effect than might be expected from the raw correlations. This is not explained by an age_group feature (no engineered band exists in the raw-only pipeline used for the final model, per Section 6.10); it more likely reflects how age correlates with the other clinical features once blood_sugar_mg_dl, cholesterol_mg_dl, and bmi are already accounted for in this linear, additive model.

10.3 Global Explanation - Feature Importance Across Classes

A bar-chart view aggregates mean absolute SHAP value per feature across all three classes at once, giving a single ranked list of which features matter most overall rather than for High risk specifically.

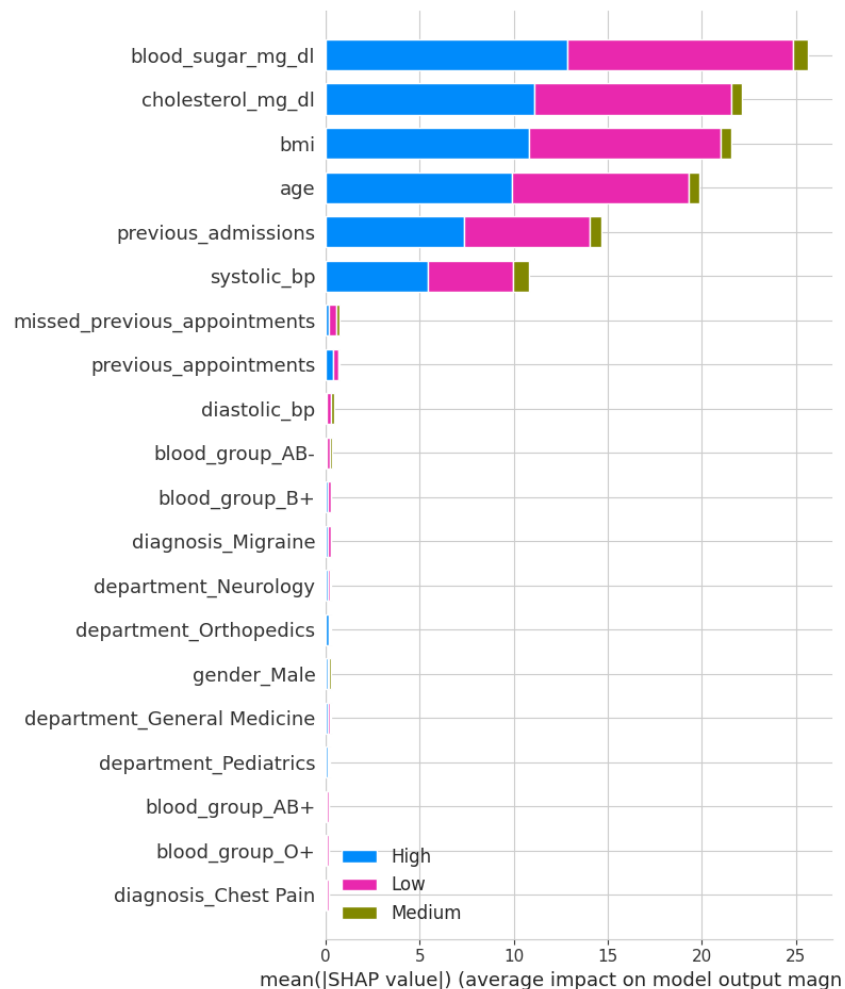


Figure 31: Mean absolute SHAP value per feature, by class.

Insight: The same five clinical features (blood_sugar_mg_dl, cholesterol_mg_dl, bmi, age, previous_admissions) dominate across all three risk classes, with blood_sugar_mg_dl carrying the strongest overall influence. The Medium class shows visibly smaller SHAP contributions than Low or High, suggesting the model finds it easier to separate the two extremes than to pin

down the middle category, consistent with Medium having the most confusion-matrix overlap with its neighbors. Engineered category and administrative features contribute comparatively little.

10.4 Local Explanations -Individual Predictions

Waterfall plots were generated for three specific patients: one correctly predicted as High risk, one correctly predicted as Low risk, and one misclassified case, showing exactly which features pushed that one patient's prediction away from the model's average output toward its final predicted class.

Correctly predicted High → predicted: High, actual: High

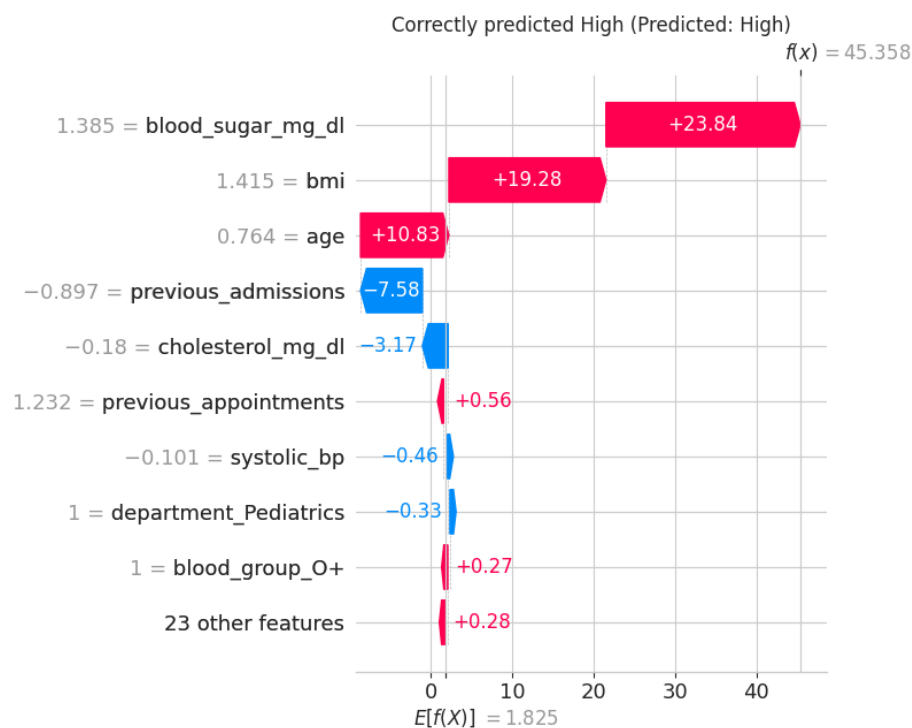


Figure 32: Waterfall — correctly predicted High-risk patient.

Correctly predicted Low → predicted: Low, actual: Low

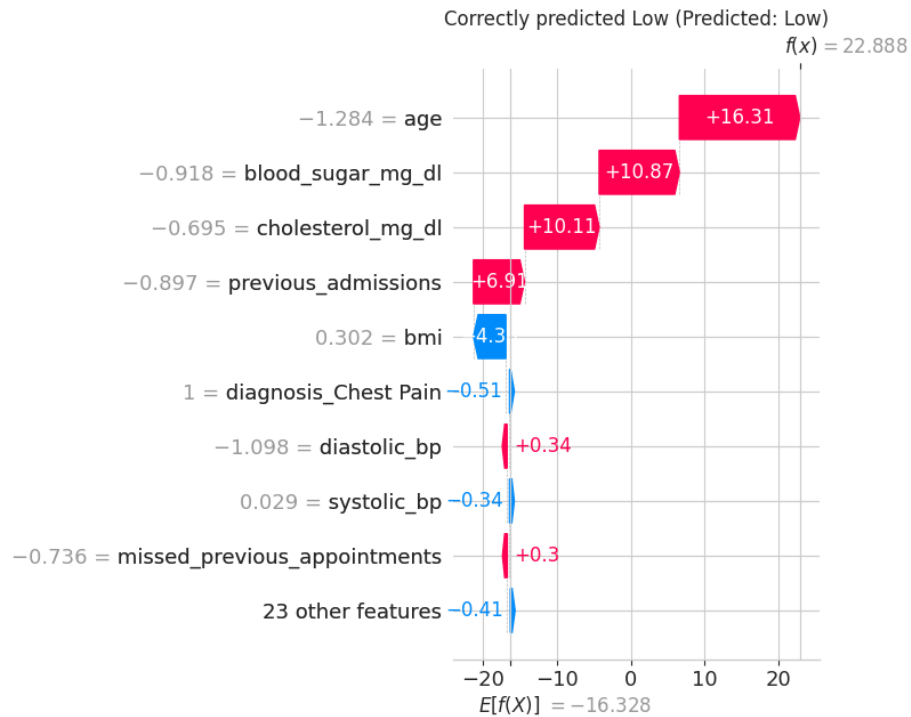


Figure 33: Waterfall — correctly predicted Low-risk patient.

Misclassified case → predicted: Medium, actual: High

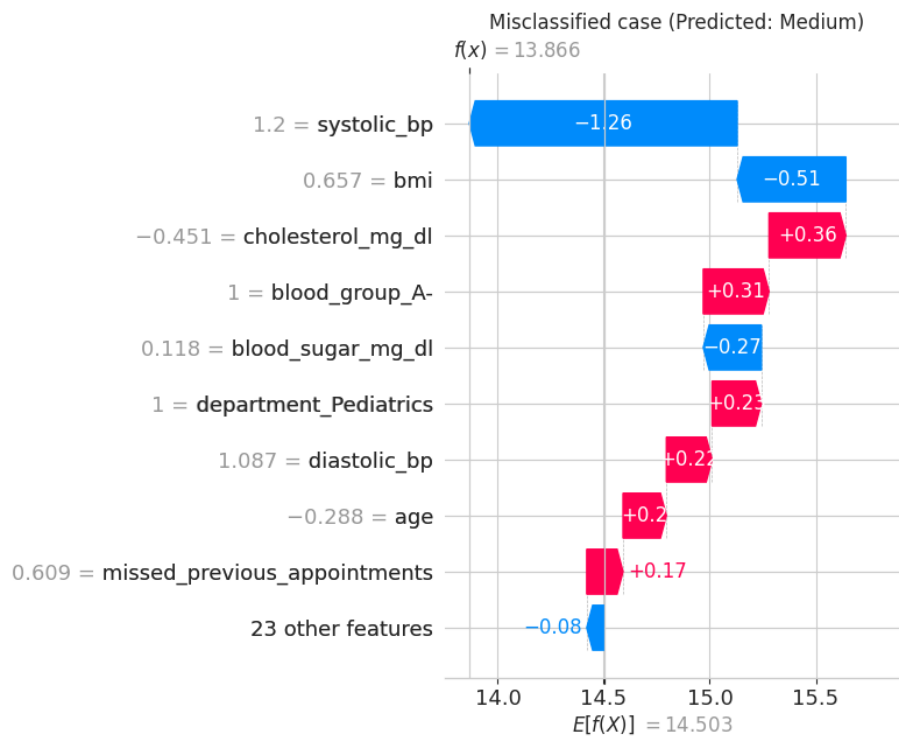


Figure 34: Waterfall — misclassified case (true: High, predicted: Medium).

Insight: For correctly predicted cases, the contributing features usually agree with each other for example, younger age and low blood sugar both support a Low-risk prediction. In the misclassified case, the features actively conflict: high BMI and cholesterol push toward High risk, while the patient's lower age and blood sugar pull toward Medium. That internal disagreement between features is itself the explanation for why the model got this one wrong; it was not a confident, clean call.

10.5 SHAP Dependence Plots - Feature Interaction Effects

Dependence plots show how a single feature's effect on the High-risk prediction changes across its range, and colour each point by a second feature to reveal interaction effects whether two variables reinforce or offset each other, something a beeswarm or bar plot alone cannot show.

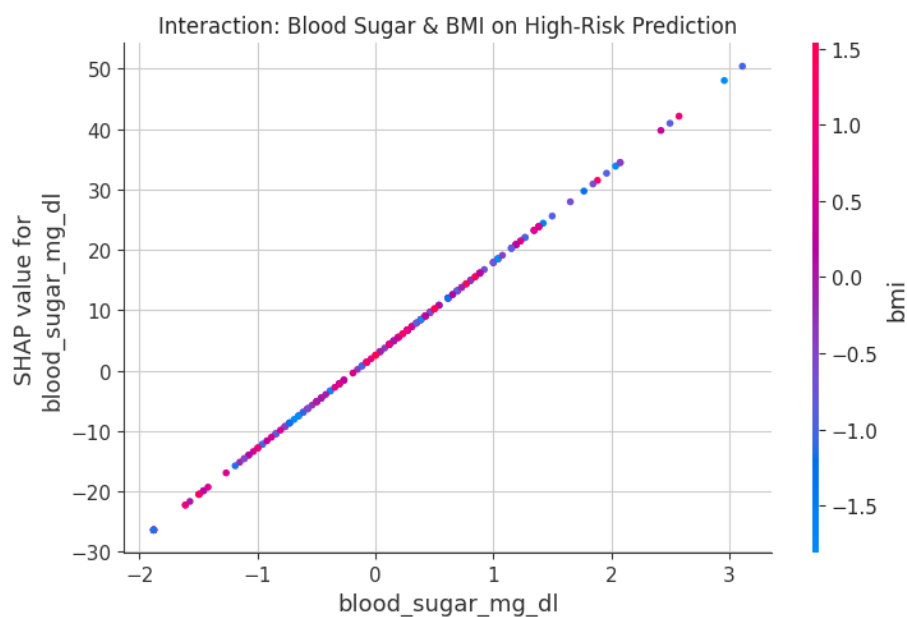


Figure 35: Interaction — blood sugar and BMI on High-risk prediction.

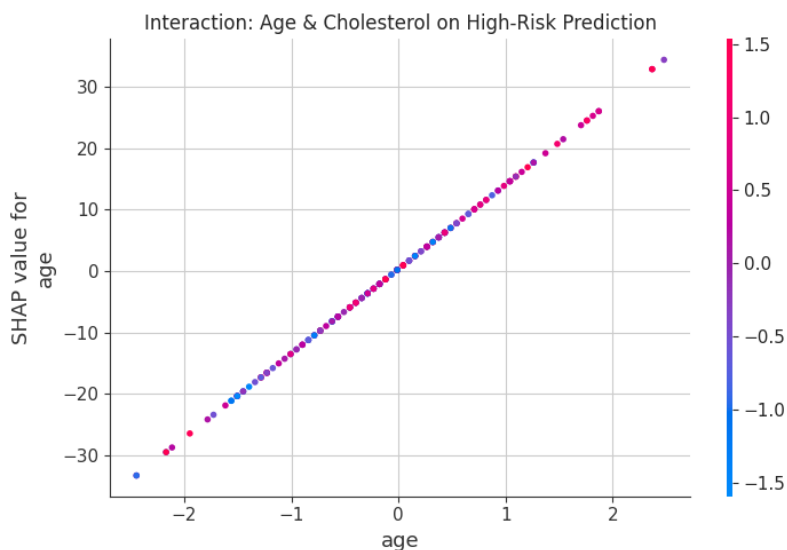


Figure 36: Interaction — age and cholesterol on High-risk prediction.

Insight: Both plots show a near-linear relationship with no clear interaction effects. BMI does not alter the blood sugar trend, and cholesterol does not change the age trend. This is expected because Logistic Regression is a linear, additive model. SHAP reflects the model's independent feature contributions rather than feature combinations. Detecting interactions would require engineered interaction terms or a tree-based model.

10.6 LIME - Cross-Validation of SHAP Explanations

LIME was run on the same misclassified case used in the waterfall plot, to check whether an independent, differently-built explanation method agrees with what SHAP reported for that patient.

SHAP and LIME reached the same conclusion for this case, showing that both explanation methods are consistent with each other — though this only confirms that the two methods agree on what the model learned, not that these are necessarily the true underlying clinical causes of the patient's risk.

10.7 Interpretation and Ethical Implications

- **Transparency:** Logistic Regression is interpretable, and SHAP's LinearExplainer provides exact feature contributions, allowing clinicians to see which patient measurements influenced each prediction.
- **Trust and error auditing:** The misclassified case was a borderline prediction (67% High vs. 33% Medium), showing that confidence scores help clinicians distinguish uncertain cases from confident predictions.
- **Ethical implications:** The model should be used as a decision-support tool, helping clinicians identify high-risk patients while leaving final decisions to clinical judgment.
- **Fairness check:** Macro-F1 was consistent across gender (Male: 0.970, Female: 0.954) and age groups (Adult: 0.941; Young Adult, Senior, Child: 1.000). The Child result is less reliable due to only 6 samples. Overall, no meaningful subgroup performance drop was observed, but small group sizes make this a preliminary fairness check.
- **Design response:** The prototype shows all three class probabilities, helping users distinguish uncertain predictions (e.g., 51% High) from confident ones (e.g., 95% High) and reducing over-trust in the model.

11. AI Prototype Development

The AI prototype packages the CV-selected explainable model (Logistic Regression, used as the interpretable proxy in Explainable AI) into a form that can accept new patient input, process it through the same preprocessing pipeline used in the preprocessing stage, and return a live disease-risk prediction together with its full class-probability breakdown.

11.1 Saving the Model Artifacts

Rather than re-deriving preprocessing logic inside the prototype, every piece needed to reproduce the preprocessing pipeline at inference time was saved directly from the training notebook: the trained model itself, the fitted ColumnTransformer that encodes raw patient fields into the same columns used for training, the manual ordinal risk mapping, the raw input columns the prototype form must collect, and the exact feature-column order the model expects.

```
joblib.dump(best_model, 'disease_risk_model.pkl')
joblib.dump(encoder_unscaled, 'encoder_unscaled.pkl')
risk_order = {'Low': 0, 'Medium': 1, 'High': 2}
joblib.dump(risk_order, 'target_encoding.pkl')
joblib.dump(FINAL_PREDICTORS, 'raw_input_columns.pkl')
joblib.dump(X_train.columns.tolist(), 'feature_columns.pkl')
```

Artifact File	Contents	Why It Is Needed
disease_risk_model.pkl	Trained Logistic Regression model (CV-selected interpretable model, Section 8/9)	Generates the actual risk prediction and class probabilities
encoder_unscaled.pkl	Fitted ColumnTransformer (scaling + one-hot encoding) from Section 6	Encodes new patient input identically to the training pipeline
target_encoding.pkl	Ordinal map {Low:0, Medium:1, High:2}	Converts the model's numeric output back to a risk label
raw_input_columns.pkl	The 13 raw predictor columns (FINAL_PREDICTORS)	Defines exactly which fields the prototype form must collect
feature_columns.pkl	Exact ordered list of the model's expected input columns	Ensures new input aligns column-for-column with what the model was trained on

Table 29: Saved artefact files and their role in the prototype.

All five artefact files were copied to the shared project drive alongside the models already saved in Section 8, so the prototype can load everything it needs without access to the original training notebook or dataset.

11.2 Working Prototype

The prototype is a Streamlit web application (prototype/app.py) built on the trained model and the five saved artefacts above. It presents the user with an input form covering the 13 raw predictor fields used in training, applies the identical encoding and scaling used in Section 6, and returns a predicted disease-risk level. Rather than showing only the predicted label, the app displays the full probability breakdown across all three classes (Low / Medium / High) — a direct response to the over-trust risk raised in Section 10.7, so a user can see how confident a given prediction actually was rather than treating every label as equally certain. The prototype can be run locally from the prototype/ folder using the command `Streamlit run app.py`.

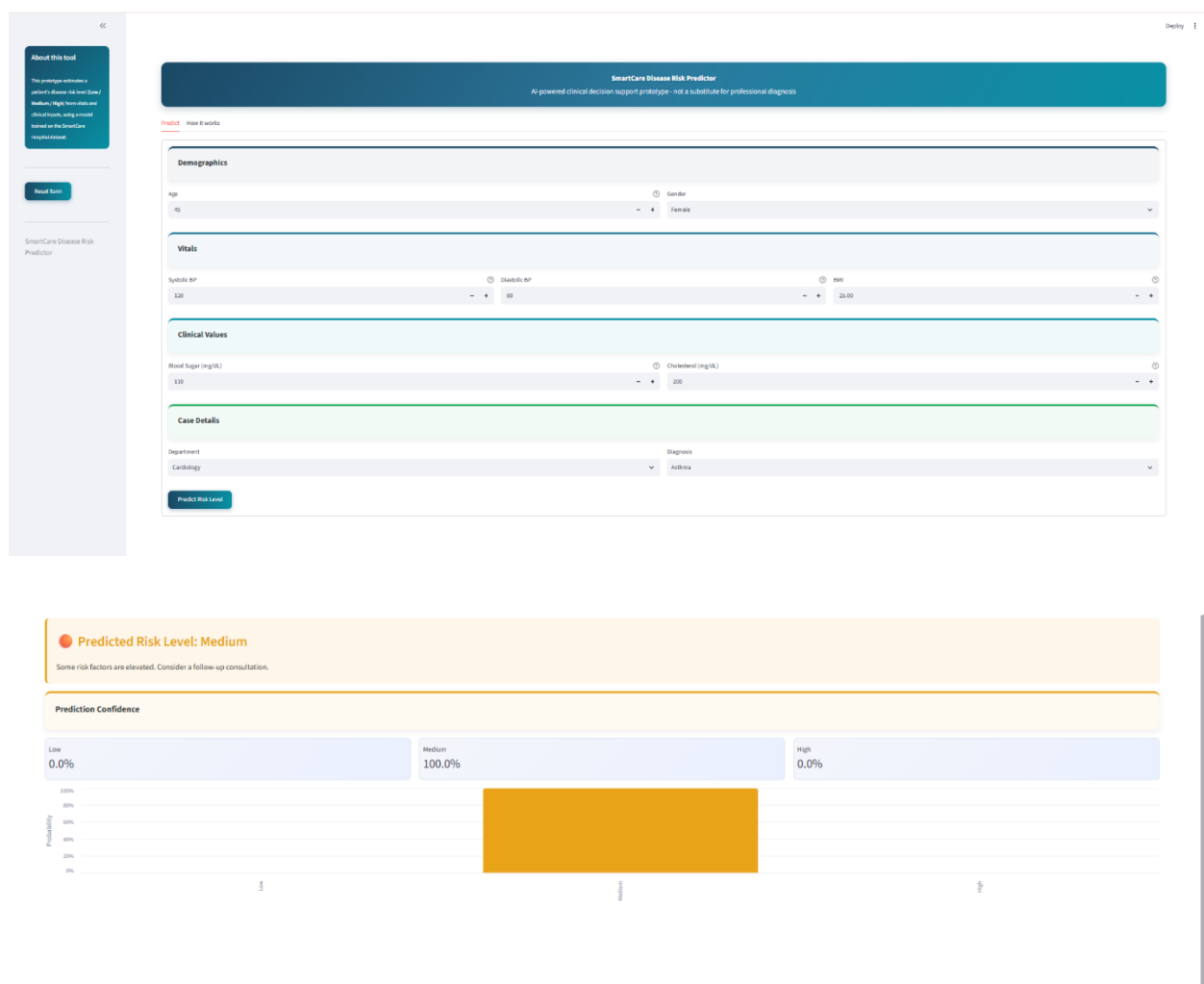


Figure 37: Prototype Design

12. Discussion and Results

Across the model pipeline, Logistic Regression (CV macro-F1 0.965; test accuracy 0.970) and the Stacking Ensemble (CV macro-F1 0.967; test accuracy 0.945) clearly outperformed the tree-based and Voting models. Because model selection was based on cross-validated macro-F1 rather than test-set results, avoiding test-set leakage, the Stacking Ensemble was selected as primary, with Logistic Regression retained for explainability. This linear dominance is explained by the EDA and SHAP analysis: engineered clinical bands (e.g., 79% of Seniors vs. 3% of Children falling High-risk) show the data's key relationships are already near-threshold rather than gradual, leaving little for tree-based flexibility to add on a small 800-row training set. A target-leakage check further confirmed this: a naive zero-ML threshold rule reached only 59.1% accuracy (0.541 macro-F1), far below the trained models indicate the strong scores reflect a genuinely learnable clinical signal, not a trivial reconstructable rule.

Two limitations persist throughout: the minority Low-risk class (13.1%) was hardest to classify across every model, justifying the use of macro-averaged metrics and confirmed stable via repeated CV (Low-class F1 = 0.949 ± 0.011 , 95% CI); and while calibration (Brier score 0.0243) and fairness checks across gender/age were encouraging, small subgroup sizes mean these remain preliminary rather than definitive. Overall, the model is a credible, well-validated decision-support tool for hospital triage best used alongside, not in place of, clinical judgement, with attention to how confidently a High-Risk label is reported.

13. Conclusion

This project develops an explainable multi-class classifier for SmartCare Hospital disease-risk prediction, covering the full AI lifecycle from data cleaning to a deployable Streamlit prototype. Logistic Regression and a Stacking Ensemble emerged as the leading models (CV macro-F1 of 0.965 and 0.967), outperforming tree-based alternatives due to sharp, near-linear age-related thresholds in the data that left little room for added complexity to help on just 800 training rows. SHAP and LIME agreed that blood sugar, cholesterol, BMI, age, and previous admissions were the primary drivers across all three classes, and a leakage investigation confirmed the model reflects a genuine clinical pattern rather than a reconstructable label rule. The main limitation is the Low-risk class (13.1%), hardest to classify throughout but is addressed via macro-averaged metrics and confirmed stable across repeated CV. With only 1,000 records from one simulated hospital, results are proof-of-concept rather than deployment-ready.

14. References

- [1]. Zaferani, N., Afrash, M. R., & Moulaei, K. (2025). Predicting and classifying type 2 diabetes using a transparent ensemble model combining random forest, k-nearest neighbor, and neural networks. *Scientific Reports*, 15, Article 31562. <https://doi.org/10.1038/s41598-025-31562-5>
- [2]. Li, X., Ding, F., Zhang, L., Zhao, S., Hu, Z., Ma, Z., Li, F., Zhang, Y., Zhao, Y., & Zhao, Y. (2025). Interpretable machine learning method to predict the risk of pre-diabetes using a national-wide cross-sectional data: Evidence from CHNS. *BMC Public Health*, 25, Article 1145. <https://doi.org/10.1186/s12889-025-22419-7>
- [3]. Zhang, L., Wang, Y., Niu, M., Wang, C., & Wang, Z. (2020). Machine learning for characterizing risk of type 2 diabetes mellitus in a rural Chinese population: The Henan Rural Cohort Study. *Scientific Reports*, 10, Article 4406.
- [4]. Dinh, A., Miertschin, S., Young, A., & Mohanty, S. D. (2019). A data-driven approach to predicting diabetes and cardiovascular disease with machine learning. *BMC Medical Informatics and Decision Making*, 19, Article 211.
- [5]. Uddin, K. M. M., Ripa, R., Yeasmin, N., Biswas, N., & Dey, S. K. (2023). Machine learning-based approach to the diagnosis of cardiovascular vascular disease using a combined dataset. *Intelligence-Based Medicine*, 7, Article 100100.
- [6]. Valencia, M. N., Kim, J., Abbas, Z., & Lee, S. W. (2026). Early detection of chronic kidney disease in men using lifestyle and demographic indicators: A machine learning approach for primary healthcare settings. *Healthcare*, 14(3), Article 405.
- [7]. He, Y., Cao, Y., Chen, Z., Xiang, R., & Wang, F. (2026). Integrating body composition analysis and machine learning for non-invasive identification of metabolic dysfunction-associated fatty liver disease: A large-scale health examination-based study. *Scientific Reports*, 16, Article 7038.
- [8]. Yang, S., Mo, R., Wang, W., Zhen, P., & Han, W. (2026). Development of a machine learning-based model for prediction of diabetes risk in patients with metabolic dysfunction-associated steatotic liver disease (MASLD). *BMJ Open*, 16(1), e107239. <https://doi.org/10.1136/bmjopen-2025-107239>